RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105

RAJALAKSHMI ENGINEERING COLLEGE

CS23334 Fundamentals of Data Science

Laboratory Record Note Book

Name : ROSHINI. N. G

Year / Branch / Section : II / CSE / I'-sec

University Register No. : 211625070161.1

College Roll No. : 250701611

Semester : III

Academic Year : 2026-2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 Fundamentals of Data Science

NAME	ROSHINI N G
ROLL NO	250401611
YEAR	II Year
SEC	I

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ROSHINI. N. G

Academic Year: 2026 Semester: III Branch: CSE

Register No. 211625070161l

Certified that this is the bonafide record of work done by the above student in the CS23334-Fundamentals of Data Science *Laboratory during the academic year 2025 – 2026.*

Signature of Faculty in-charge

Submitted for the Practical Examination held on .....

Internal Examiner External Examiner

INDEX

SlNO	Experiment	Date	GitHub
1	Installation of Open CV & To perform the basic image handling processing operation on the image	03/01/2026	
2	Implement Edge Detection, Line Detection and Corner Detection	10/01/2026	
3	Demonstrate Camera Calibration using python	24/01/2026	
4	Implement Image Histogram and Histogram Equalization	31/01/2026	
5	Develop a python program for Skin color Detection	07/02/2026	
6	Create a python program for Warping and Estimation	14/02/2026	
7	Develop a python program for Motion Tracking	21/02/2026	
8	Design a program for Object Detection using YOLO	07/03/2026	
9	Develop a python program for Stereo Vision and Depth Estimation	14/03/2026	
10	Demonstrate Augmented reality using feature matching	04/04/2026	
11	Mini Project <project title>	11/04/2026, 18/04/2026, 25/04/2026, 02/05/2026	

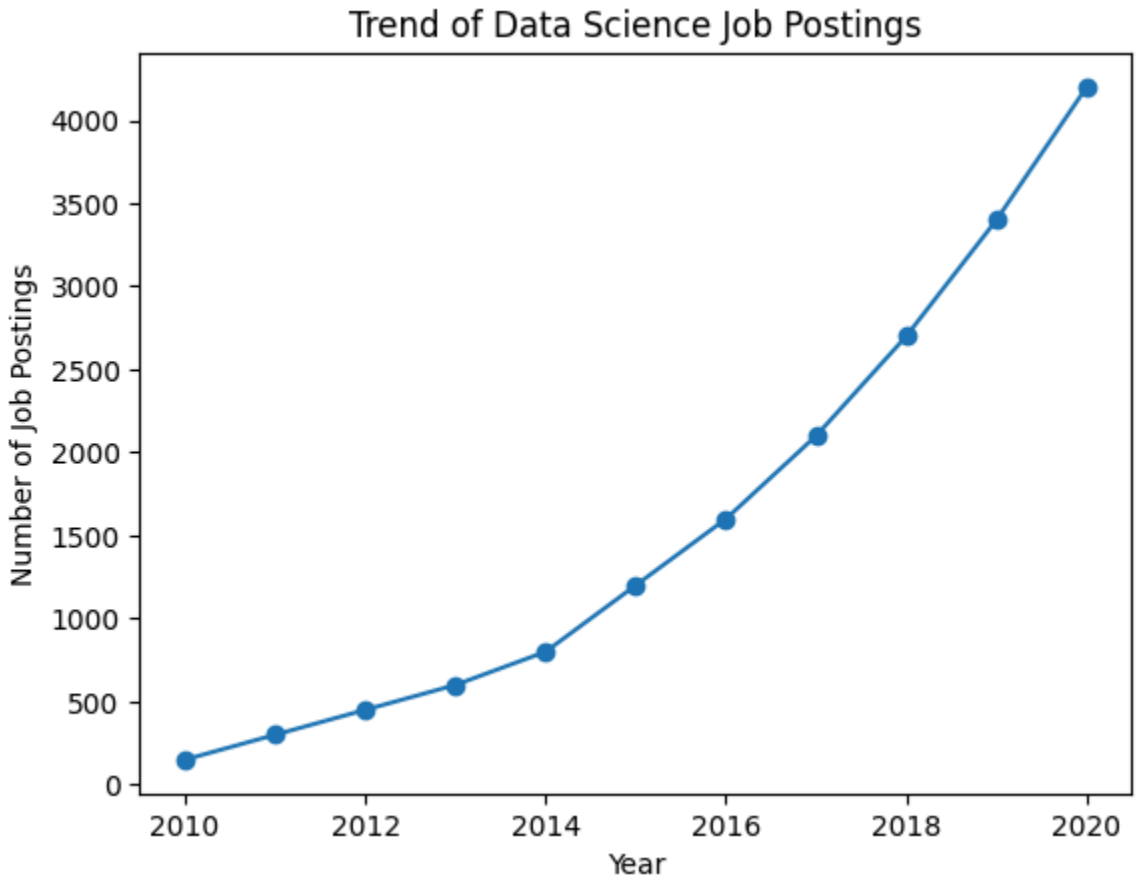
EXPERIMENT NO: 1(a)

Analyse the trend of data science job postings over the last decade

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
data={'Year': list(range(2010,2021)),
      'Job Postings': [150,300,450,600,800,1200,1600,2100,2700,3400,4200]}
df=pd.DataFrame(data)
plt.plot(df['Year'],df['Job Postings'],marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.show()
```

Output:



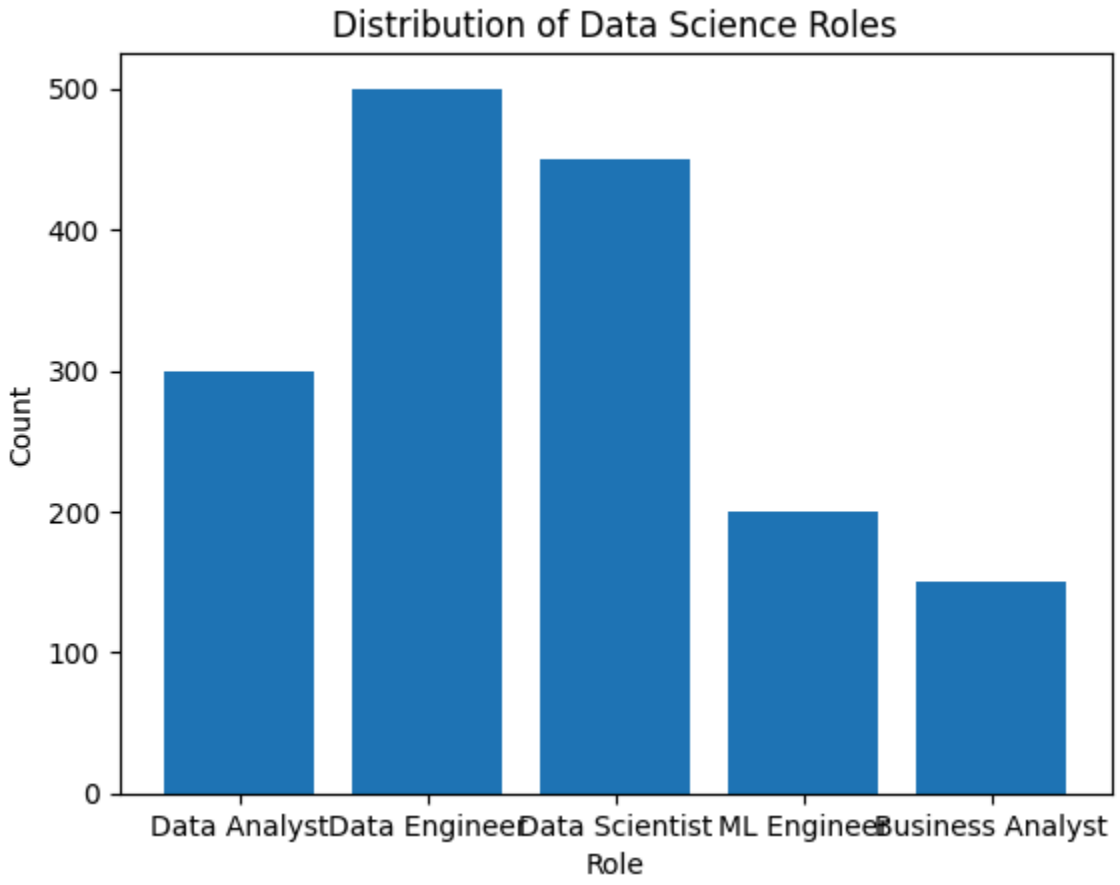
EXPERIMENT NO: 1(b)

Analyse and Visualize the Distribution of various Data Science Roles from a Dataset

Program:

```
import matplotlib.pyplot as plt
roles=['Data Analyst','Data Engineer','Data Scientist','ML Engineer',
'Business Analyst']
counts=[300,500,450,200,150]
plt.bar(roles,counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```

Output:



EXPERIMENT NO: 1(c)

Conduct an Experiment to differentiate datas based on data sets given

Program:

```
#Structured data
Structured_data=pd.DataFrame({
    'ID':[1,2,3],
    'Name':['Alice','Bob','Carlie'],
    'Age':[25,30,35]
})
print("Structured Data:\n",Structured_data)

#Unstructured data
unstructured_data="This is an example of Unstructured data. It can be a
piece of text, an image, or a video file."
print("\n Unstructured Data:\n",unstructured_data)

#Semi-structured data
semi_structured_data={'ID':1,'Name':'Alice','Attributes':{'Height':165,'We
ight':68}}
print("\nSemi-Structured Data:\n",semi_structured_data)
```

Output:

Structured Data:

	ID	Name	Age
0	1	Alice	25
1	2	Bob	30
2	3	Carlie	35

Unstructured Data:

This is an example of Unstructured data. It can be a piece of text, an image, or a video file.

Semi-Structured Data:

{'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}

EXPERIMENT NO: 1(d)

Conduct an experiment to encrypt and decrypt given sensitive data

Program:

```
#Generate key and encrypt data
from cryptography.fernet import Fernet
key=Fernet.generate_key()
f=Fernet(key)
token=f.encrypt(b"Rajalakshmi Engineering College")
token
b'....'
f.decrypt(token)
b'Rajalakshmi Engineering College'
key=Fernet.generate_key()
cipher_suite=Fernet(key)
plain_text=b'Rajalakshmi Engineering College.'
cipher_text=cipher_suite.encrypt(plain_text)

#Decrpyt Data
decrypted_text=cipher_suite.decrypt(cipher_text)
print("Original Data: ",plain_text)
print("Encrpyted Data: ",cipher_text)
print("Decrpyted Data: ",decrypted_text)
```

Output:

Original Data: b'Rajalakshmi Engineering College.'

Encrpyted Data:

b'gAAAAABqoVa79_0dbI4tCrFzYT9-xcnJXcWvARfZYAHgo6lJmDPMYVvLf4dNCocedgX1zeUXov6H1EpfOsnQpP91NZwwbjORnRbXHlWP9CWeyiMBS8DFL0h-QG-X0vk3JHCsJ09btGUA'

Decrpyted Data: b'Rajalakshmi Engineering College.'

EXPERIMENT NO: 02

Upload and Analyze the data set given in CSV Format and Perform Data Preprocessing and Visualization

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
#Load the data into a pandas DataFrame
file_path='Sales_data.csv'
df=pd.read_csv(file_path)
#Display the first few rows of the Dataframe
print(df.head())
#Check for missing values
print(df.isnull().sum())
#Fill or drop missing values if necessary
df['Sales'].fillna(df['Sales'].mean(),inplace=True)
df.dropna(subset=['Product','Quantity','Region'],inplace=True)
#Summary Statistics
print(df.describe())
#Group by product and calculate the total sales and quantity
product_summary=df.groupby('Product').agg({
    'Sales':'sum',
    'Quantity':'sum'
}).reset_index()
print(product_summary)
#Bar plot of total sales by product
plt.figure(figsize=(10,6))
plt.bar(product_summary['Product'],product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

#line plot of sales over time
df['Date']=pd.to_datetime(df['Date'])
sales_over_time=df.groupby('Date').agg({
    'Sales':'sum'
```

```
}).reset_index()
plt.figure(figsize=(10,6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()

#Pivot table to analyze sales by region and product
pivot_table=df.pivot_table(values='Sales',index='Region',columns='Product'
,aggfunc=np.sum,fill_value=0)
print(pivot_table)
#correlation matrix
correlation_matrix=df.corr(numeric_only=True)
print(correlation_matrix)

#heatmap of the correlation matrix
plt.figure(figsize=(8,6))
sns.heatmap(correlation_matrix,annot=True,cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

Output:

	Date	Product	Sales	Quantity	Region
0	2023-01-01	Product A	200	4	North
1	2023-01-02	Product B	150	3	South
2	2023-01-03	Product A	220	5	North
3	2023-01-04	Product C	300	6	East
4	2023-01-05	Product B	180	4	West

```
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
```

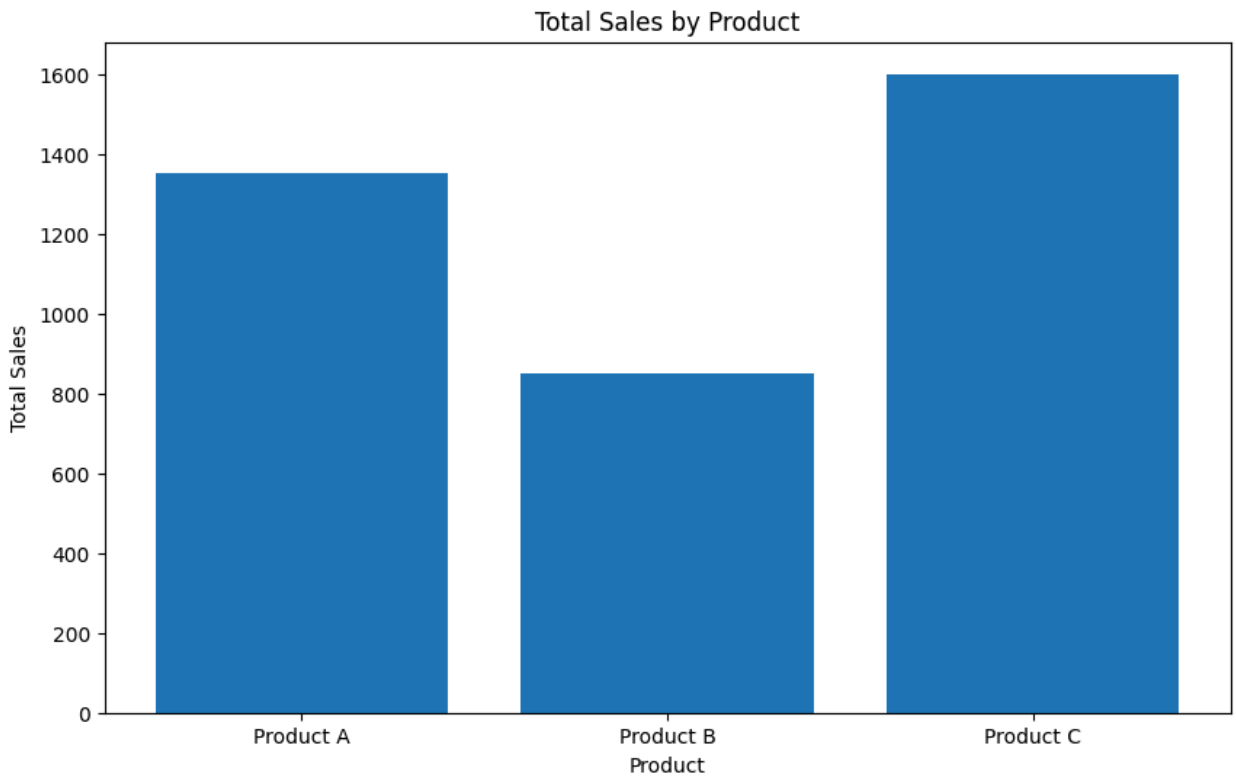
	Sales	Quantity
count	16.000000	16.000000

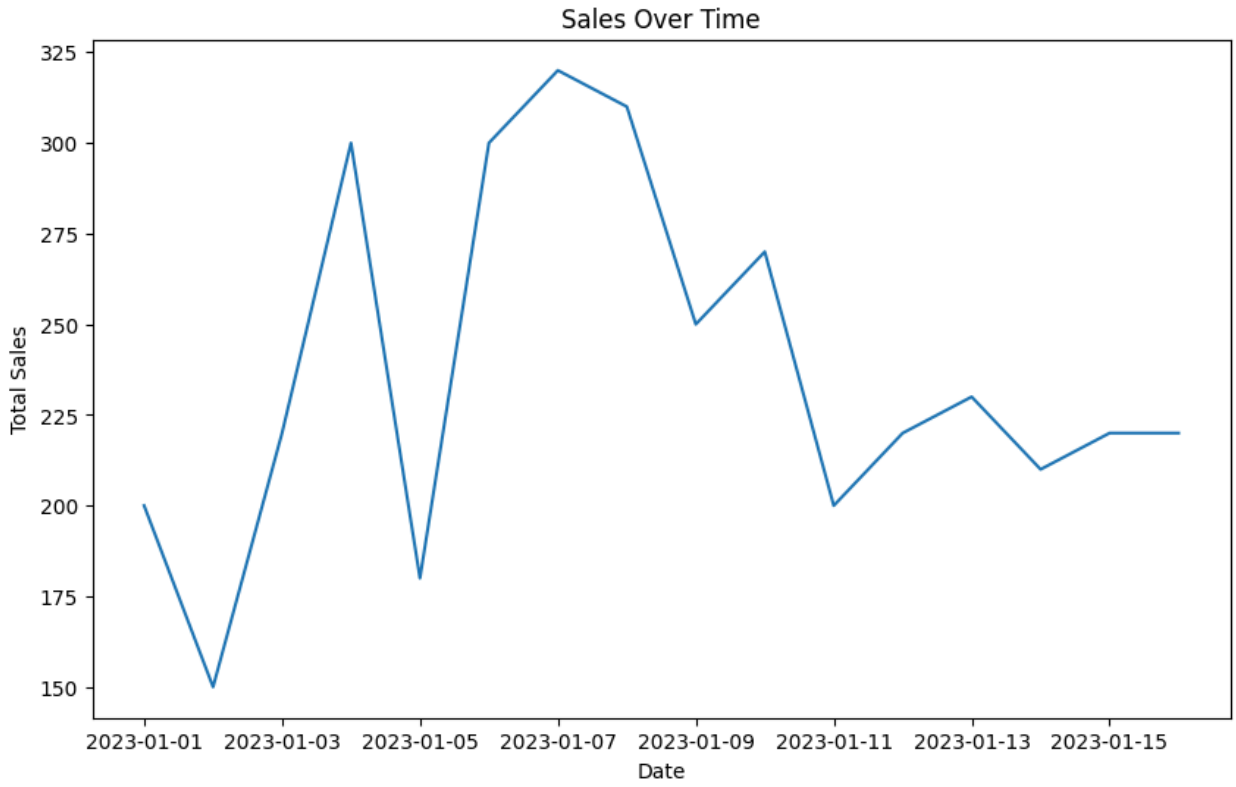
```
mean   237.500000   5.375000
std     49.598387   1.454877
min    150.000000   3.000000
25%    207.500000   5.000000
50%    220.000000   5.000000
75%    277.500000   5.250000
max    320.000000   8.000000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```

```
/tmp/ipykernel_3443/1309037498.py:13: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
```

```
df['Sales'].fillna(df['Sales'].mean(),inplace=True)
```



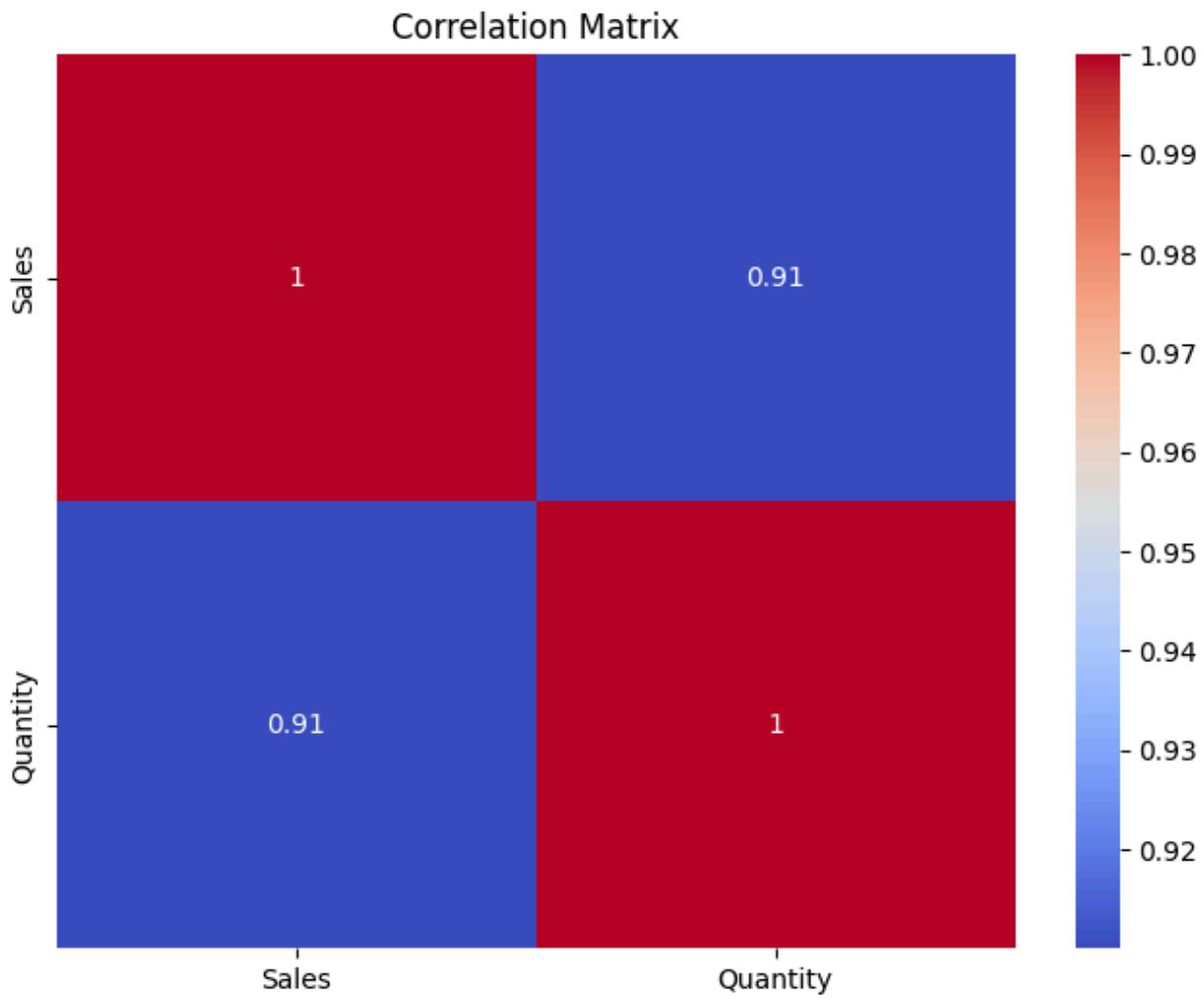


/tmp/ipykernel_3443/1309037498.py:44: FutureWarning: The provided callable <function sum at 0x7f764e80ab60> is currently using

DataFrameGroupBy.sum. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "sum" instead.

```
pivot_table=df.pivot_table(values='Sales',index='Region',columns='Product'
,aggfunc=np.sum,fill_value=0)
```

```
Product  Product A  Product B  Product C
Region
East           310          0        740
North          720          0        410
South          320        420        220
West             0        430        230
            Sales  Quantity
Sales     1.000000  0.910021
Quantity  0.910021  1.000000
```


Correlation Matrix
Sales
Quantity
1
0.91
0.91
1
Sales
Quantity
1.00
0.99
0.98
0.97
0.96
0.95
0.94
0.93
0.92

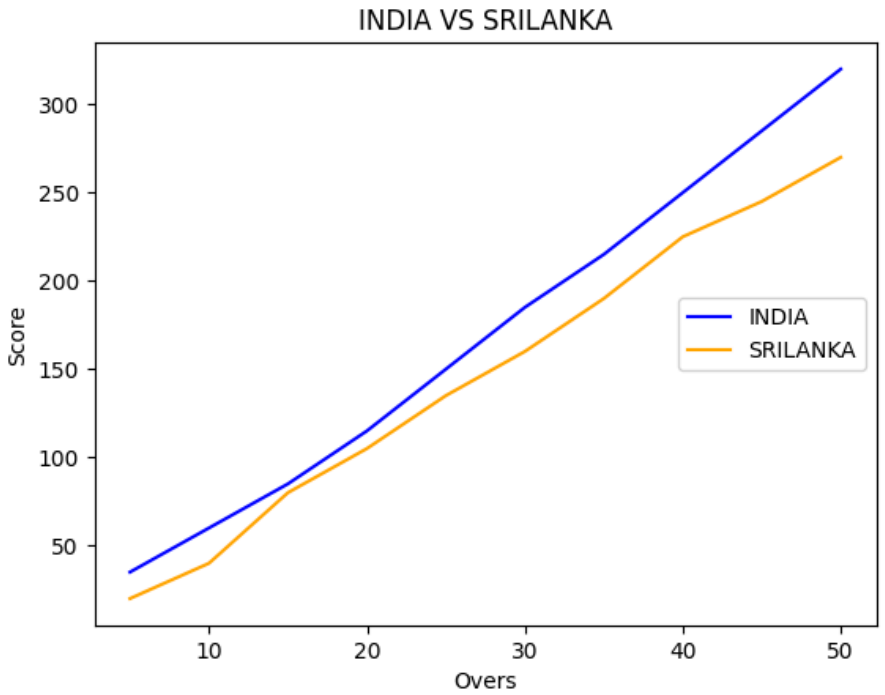
EXPERIMENT NO: 3(a)

Conduct an Experiment to show data visualization using line plot

Program:

```
import matplotlib.pyplot as cricket
overs=list(range(5,51,5))
Indian_score=[35,60,85,115,150,185,215,250,285,320]
Srilankan_score=[20,40,80,105,135,160,190,225,245,270]
cricket.plot(overs,Indian_score)
cricket.plot(overs,Srilankan_score)
cricket.show()
cricket.title("INDIA VS SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(overs,Indian_score,color="blue",label="INDIA")
cricket.plot(overs,Srilankan_score,color="orange",label="SRILANKA")
cricket.legend(loc="center right")
```

Output:



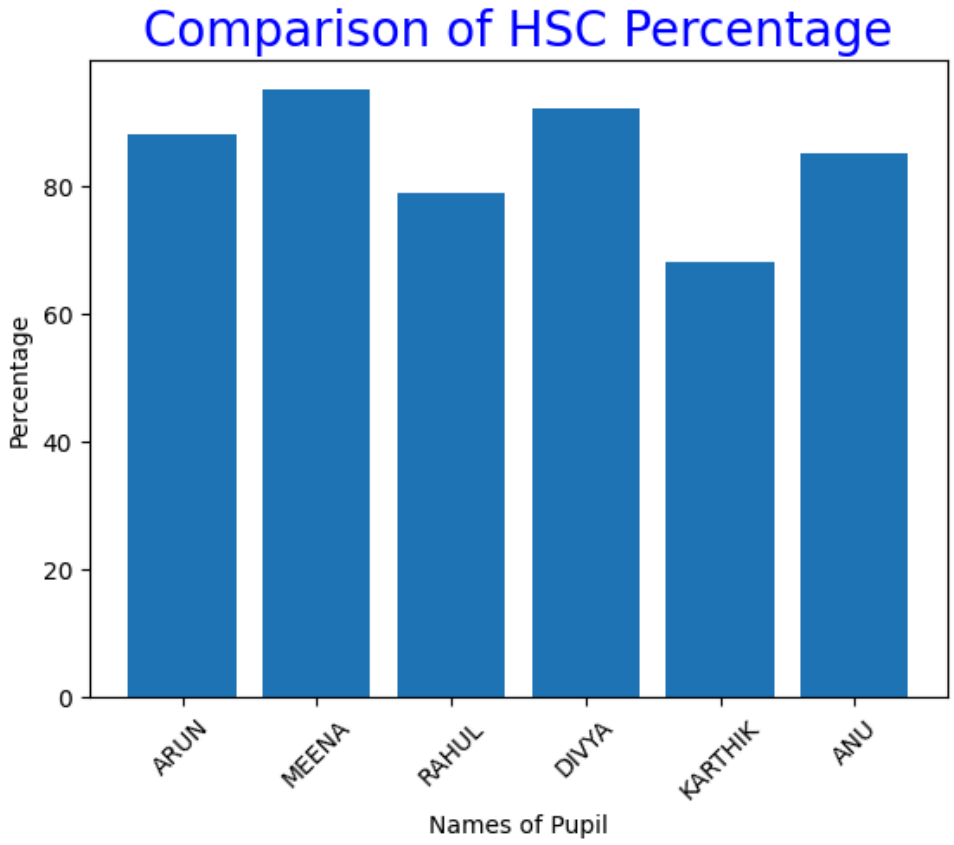
EXPERIMENT NO: 03(b)

Conduct an Experiment to show data visualization using bar chart

Program:

```
import matplotlib.pyplot as hscmark
import numpy as np
Names=['ARUN','MEENA','RAHUL','DIVYA','KARTHIK','ANU']
xaxis=np.arange(len(Names))
percentage_hsc=[88,95,79,92,68,85]
hscmark.bar(Names,percentage_hsc)
hscmark.xticks(xaxis,Names,rotation=45)
hscmark.ylabel("Percentage")
hscmark.xlabel("Names of Pupil")
hscmark.title("Comparison of HSC Percentage",fontsize=20,color="blue")
hscmark.show()
```

Output:



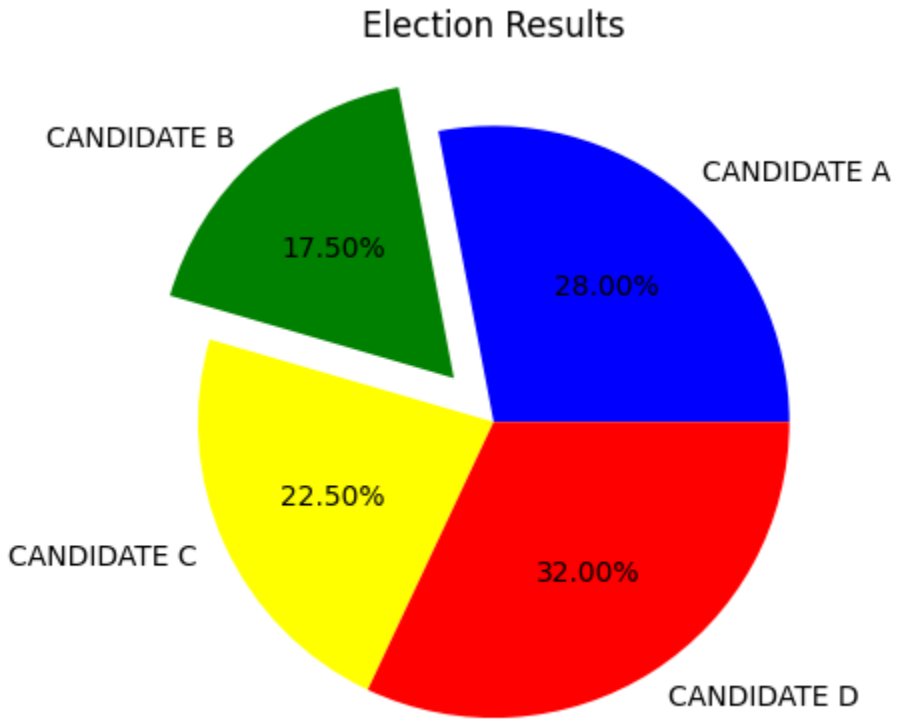
EXPERIMENT NO: 3(c)

Conduct an Experiment to show data visualization using pie chart

Program:

```
import matplotlib.pyplot as election
labels=['CANDIDATE A','CANDIDATE B','CANDIDATE C','CANDIDATE D']
votes=[280,175,225,320]
colors=['blue','green','yellow','red']
explode=(0,0.2,0,0)
election.pie(votes,labels=labels,colors=colors,explode=explode,autopct='%0
.2f%%')
election.title('Election Results')
election.show()
```

Output:



EXPERIMENT NO: 04

To count the frequency of occurrence of a word in a body of text is often needed during text processing

Program:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')
sample=gutenberg.raw("shakespeare-hamlet.txt")
token=word_tokenize(sample)
wlist=[]
for i in range(30):
  wlist.append(token[i])
wordfreq=[wlist.count(w) for w in wlist]
print("Pairs\n"+str(list(zip(wlist,wordfreq))))
```

Output:

```
[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data]   Package gutenberg is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!

Pairs
[('[', 1), ('The', 1), ('Tragedie', 1), ('of', 1), ('Hamlet', 1), ('by', 1), ('William', 1), ('Shakespeare', 1),
('1599', 1), (']', 1), ('Actus', 1), ('Primus', 1), ('.', 4), ('Scoena', 1), ('Prima', 1), ('.', 4), ('Enter', 1),
('Barnardo', 2), ('and', 1), ('Francisco', 1), ('two', 1), ('Centinels', 1), ('.', 4), ('Barnardo', 2), ('.', 4),
('Who', 1), ("'s", 1), ('there', 1), ('?', 1), ('Fran', 1)]
```

EXPERIMENT NO: 05

Data Collection and Initial Exploration

Program:

```
import pandas as pd
diabetes_df=pd.read_csv("diabetes (1).csv")
print(diabetes_df.head())
print(diabetes_df.info())
print(diabetes_df.describe())
import matplotlib.pyplot as plt
import seaborn as sns
diabetes_df.hist(bins=50,figsize=(20,15))
plt.show()
sns.pairplot(diabetes_df)
plt.show()
```

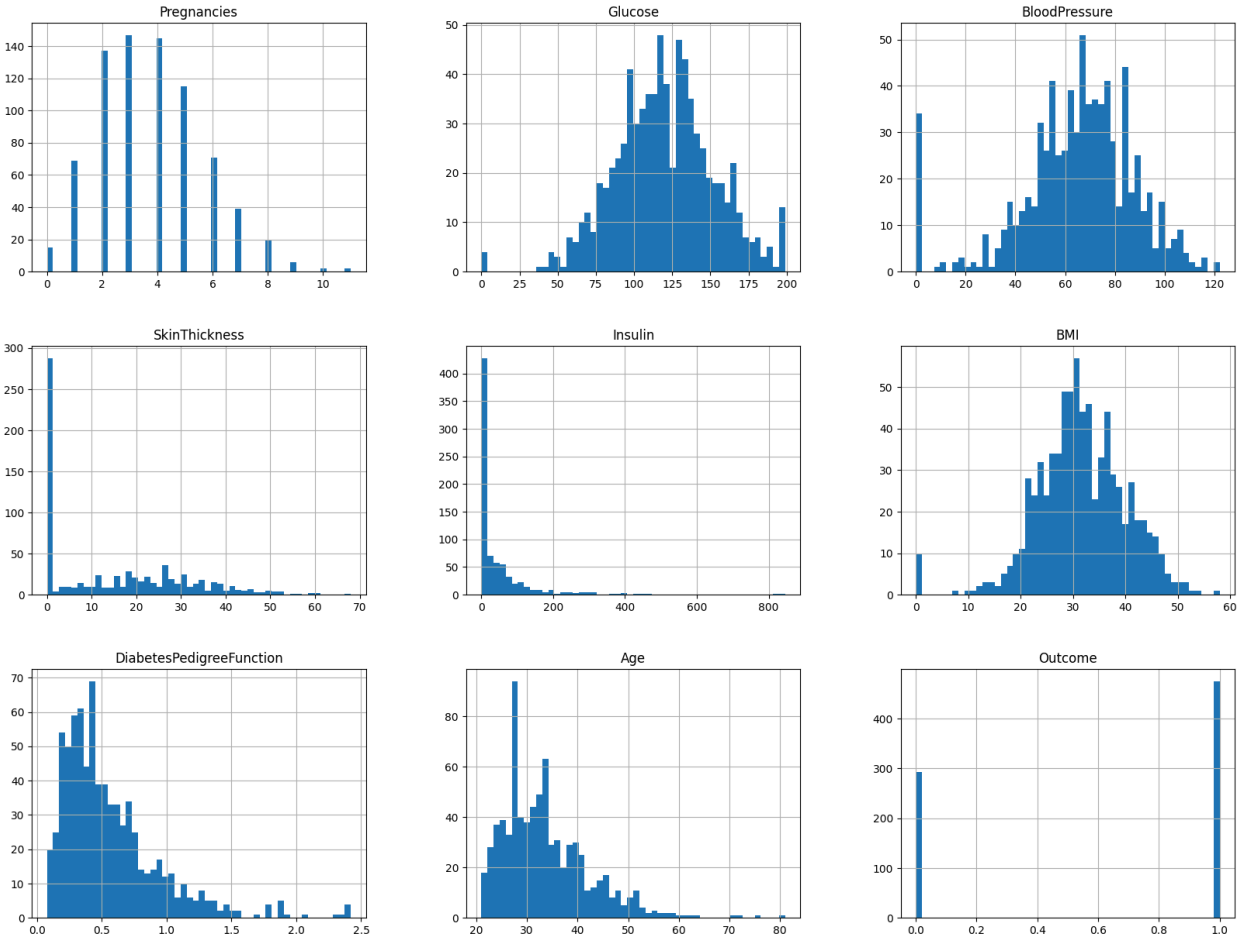
Output:

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
```

```
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.768229  120.165365      64.972656      15.522135   44.391927
std       1.946060   32.387525      23.193434      15.576602   84.220647
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       2.000000   98.000000      53.000000       0.000000    0.000000
50%       4.000000  119.000000      67.000000      13.000000    5.000000
75%       5.000000  140.000000      80.000000      27.000000   55.250000
max      11.000000  199.000000     122.000000      68.000000  846.000000

              BMI  DiabetesPedigreeFunction        Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.616276                  0.563865   33.558594    0.618490
std      8.581108                  0.384463    8.546778    0.486074
min      0.000000                  0.078000   21.000000    0.000000
25%     26.450000                  0.299750   27.000000    0.000000
50%     31.300000                  0.454500   32.000000    1.000000
75%     37.100000                  0.717000   38.000000    1.000000
max     58.000000                  2.420000   81.000000    1.000000
```


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome

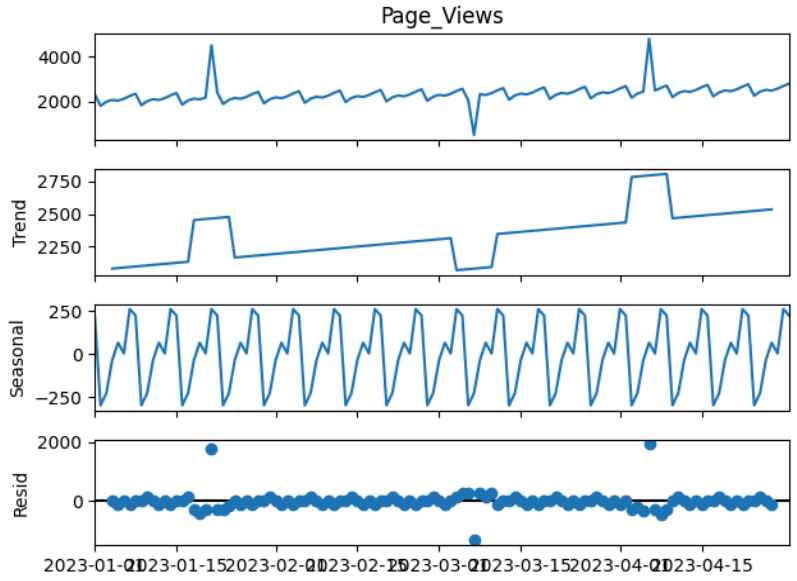
EXPERIMENT NO: 06

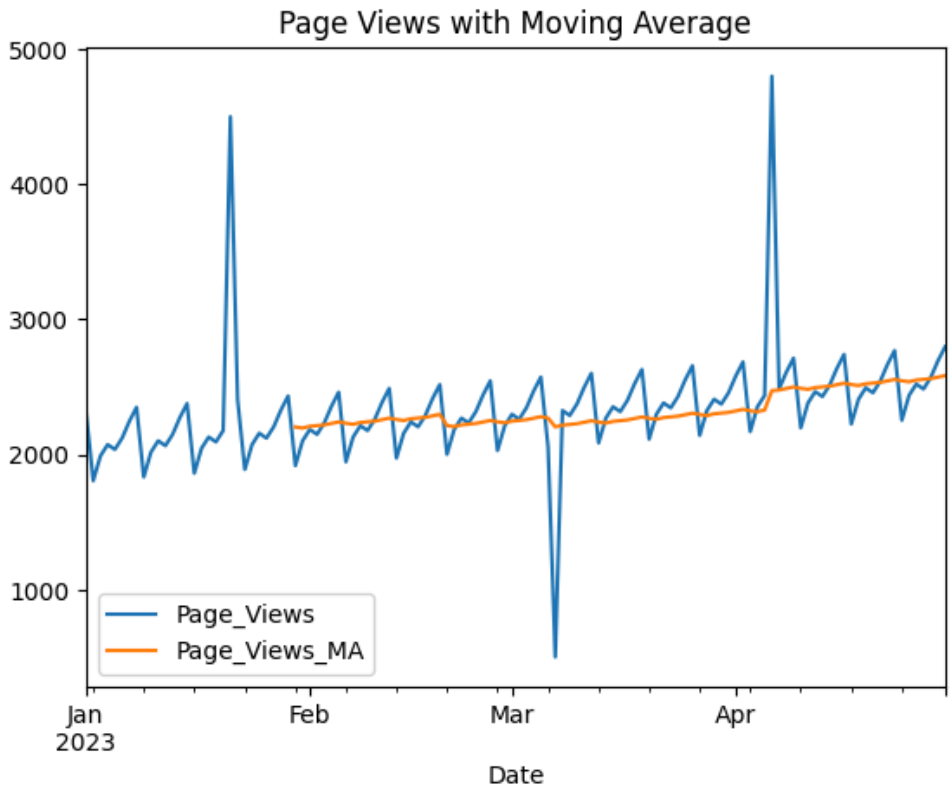
Time Series Analysis of Website Traffic

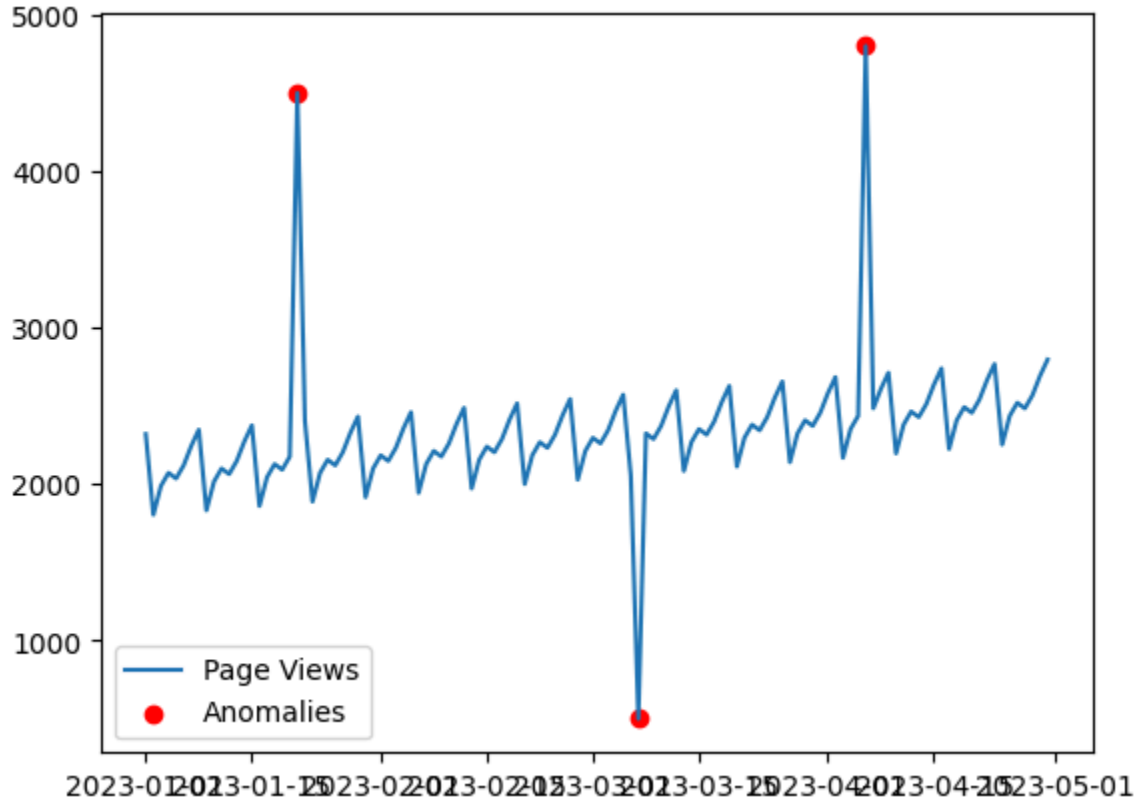
Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
data=pd.read_csv('website_traffic.csv',parse_dates=['Date'],index_col='Dat
e')
decomposition=seasonal_decompose(data['Page_Views'],model='additive')
decomposition.plot()
plt.show()
data['Page_Views_MA']=data['Page_Views'].rolling(window=30).mean()
data[['Page_Views','Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()
data['Page_Views_Z']=(data['Page_Views']-data['Page_Views'].mean())/data['
Page_Views'].std()
anomalies=data[data['Page_Views_Z'].abs()>3]
plt.plot(data.index,data['Page_Views'],label='Page Views')
plt.scatter(anomalies.index,anomalies['Page_Views'],color='red',label='Ano
malies')
plt.legend()
plt.show()
```

Output:




Page Views with Moving Average
5000
4000
3000
2000
1000
Page_Views
Page_Views_MA
Jan
2023
Feb
Mar
Apr
Date


5000
4000
3000
2000
1000
Page Views
Anomalies
2023-01-01
2023-01-15
2023-02-01
2023-02-15
2023-03-01
2023-03-15
2023-04-01
2023-04-15
2023-05-01

EXPERIMENT NO: 07

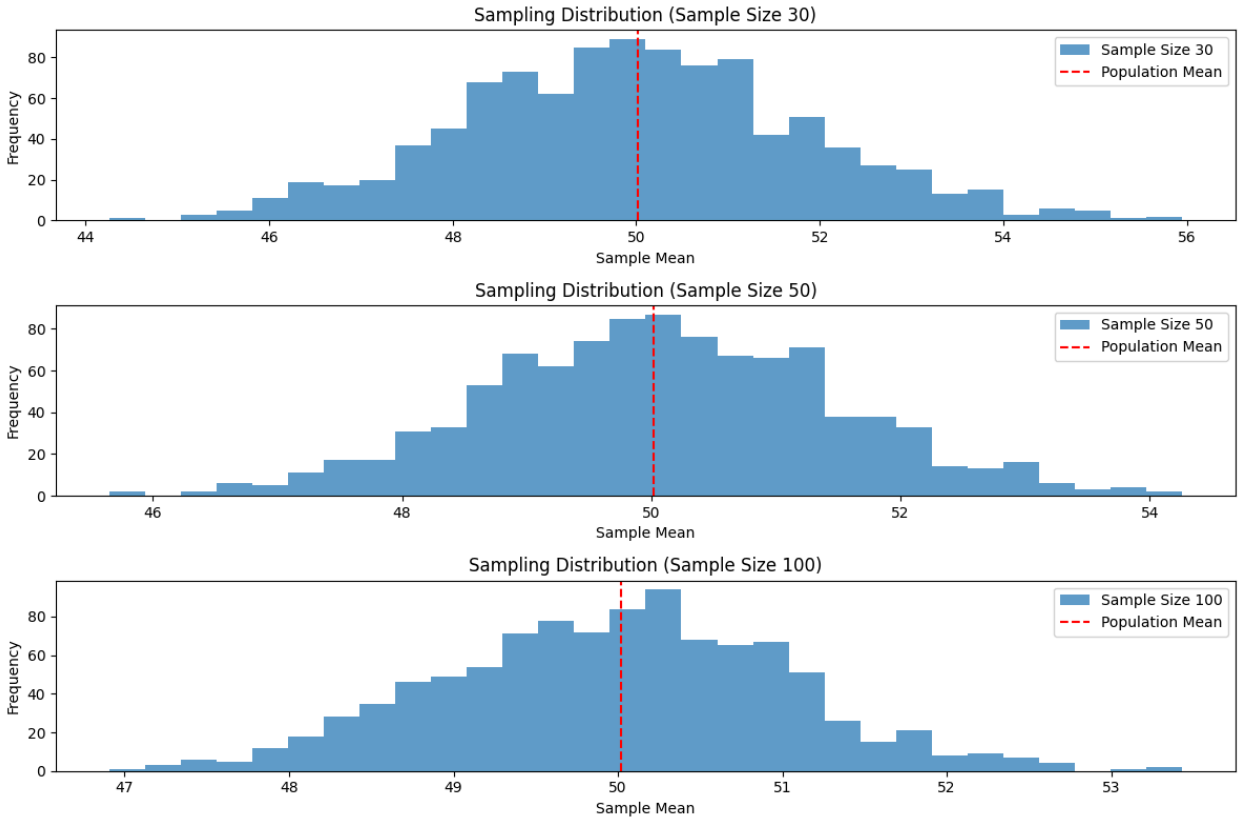
Random Sampling and Sampling Distribution

Program:

```
import numpy as np
import matplotlib.pyplot as plt
population_mean=50
population_std=10
population_size=100000
population=np.random.normal(population_mean,population_std,population_size
)
sample_sizes=[30,50,100]
num_samples=1000
sample_means={}
for size in sample_sizes:
  sample_means[size]=[]
  for _ in range(num_samples):
    sample=np.random.choice(population,size=size,replace=False)
    sample_means[size].append(np.mean(sample))
plt.figure(figsize=(12,8))
for i, size in enumerate(sample_sizes):
  plt.subplot(len(sample_sizes),1,i+1)
  plt.hist(sample_means[size],bins=30,alpha=0.7,label=f'Sample Size
{size}')

plt.axvline(np.mean(population),color='red',linestyle='dashed',linewidth=1
.5,label='Population Mean')
  plt.title(f'Sampling Distribution (Sample Size {size})')
  plt.xlabel('Sample Mean')
  plt.ylabel('Frequency')
  plt.legend()
plt.tight_layout()
plt.show()
```

Output:


Sampling Distribution (Sample Size 30)
Sample Size 30
Population Mean
Frequency
Sample Mean
Sampling Distribution (Sample Size 50)
Sample Size 50
Population Mean
Frequency
Sample Mean
Sampling Distribution (Sample Size 100)
Sample Size 100
Population Mean
Frequency
Sample Mean

EXPERIMENT NO: 08

Conduct Z test for the data given using Python Objective

Program:

```
import numpy as np
import scipy.stats as stats
sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152,
149,151, 150, 149, 152, 151,
148, 150, 152, 149, 150,148, 153, 151, 150, 149, 152, 148, 151, 150, 153])
population_mean = 150
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)
n = len(sample_data)
z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))
p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: The average weight is significantly
different from 150 grams.")
else:
    print("Fail to reject the null hypothesis: There is no significant
difference in average weight from 150 grams.")
```

Output:

Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant difference in average weight from 150 grams.

EXPERIMENT NO: 09

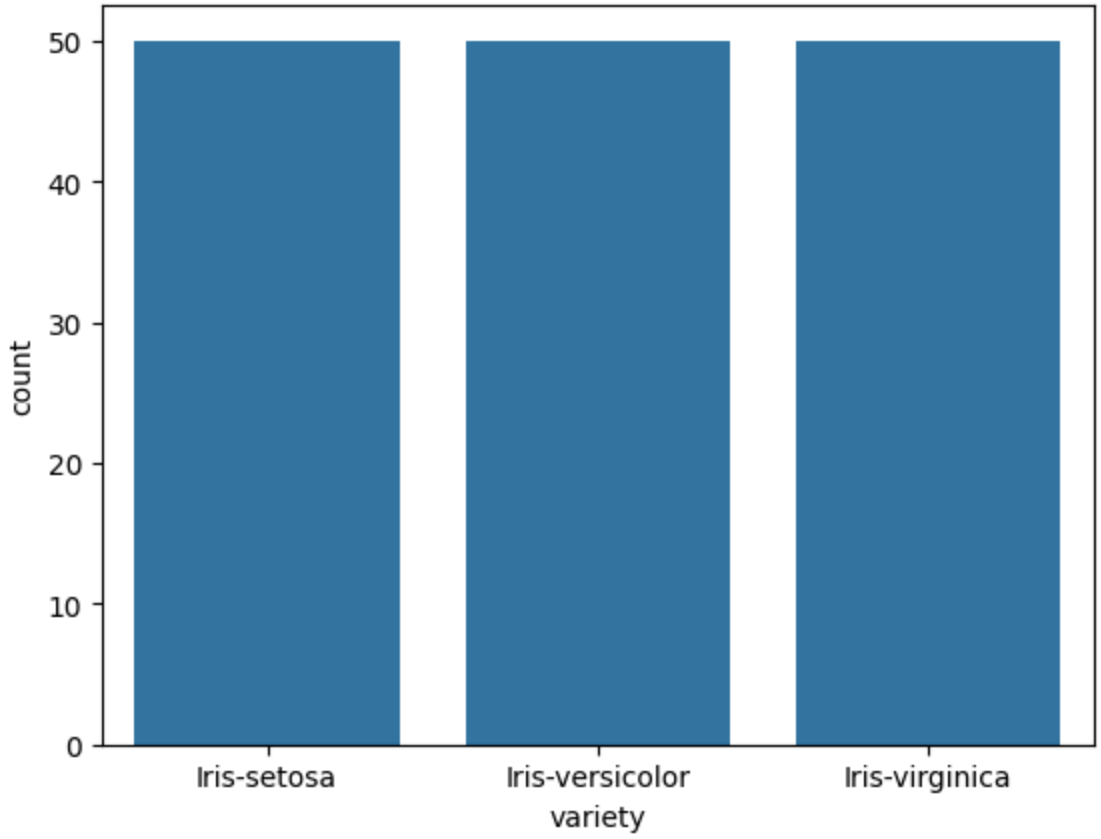
Experiment to understand Matplotlib library use cases in Data Science through visualization

Program:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
data=pd.read_csv('/content/Iris_Dataset.csv')
data.info()
print(data.describe())
print(data.value_counts('variety'))
sns.countplot(x='variety',data=data,)
plt.show()
dummies=pd.get_dummies(data.variety)
FinalDataset=pd.concat([pd.get_dummies(data.variety),data.iloc[:,[0,1,2,3]
]],axis=1)
print(FinalDataset.head())
sns.scatterplot(x='SepalLengthCm',y='SepalWidthCm',hue='variety',data=data
,)
sns.scatterplot(x='PetalLengthCm',y='PetalWidthCm',hue='variety',data=data
,)
sns.pairplot(data,hue='variety',height=3);
plt.show()
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'PetalLengthCm
').add_legend();
plt.show();
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'PetalWidthCm'
).add_legend();
plt.show();
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'SepalLengthCm
').add_legend();
plt.show();
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'SepalWidthCm'
).add_legend();
plt.show();
```

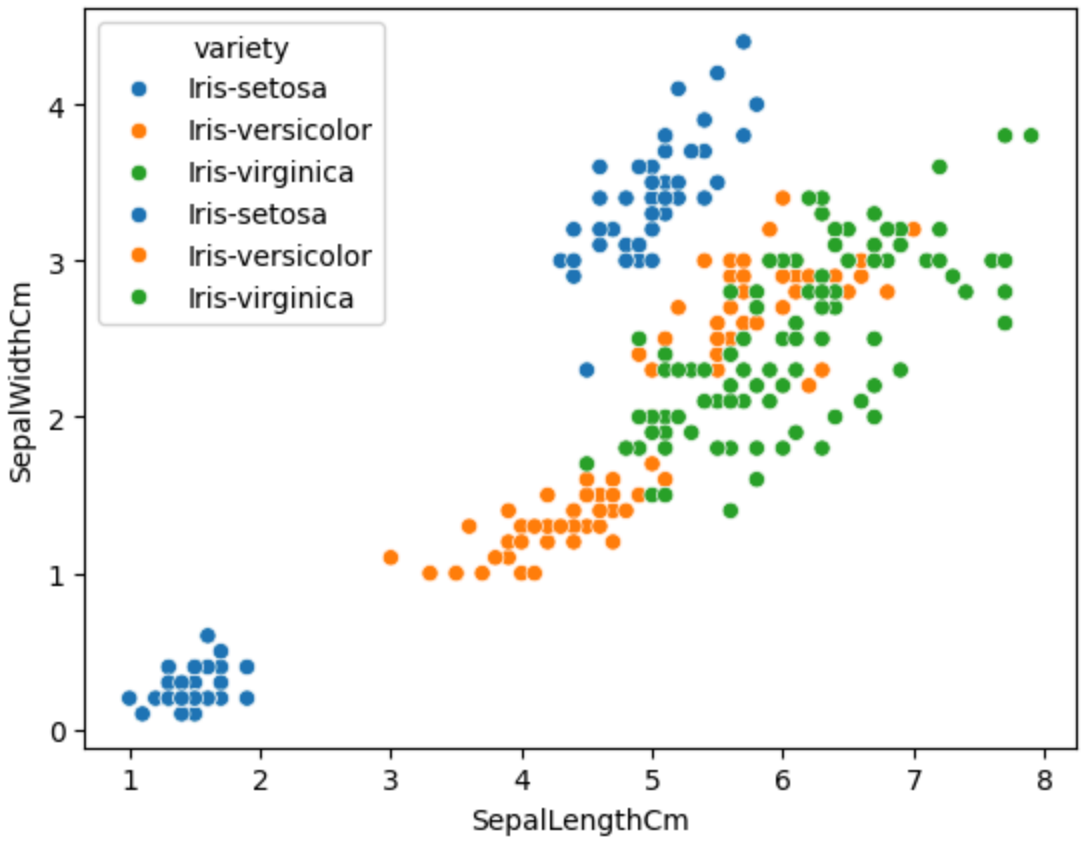
Output:

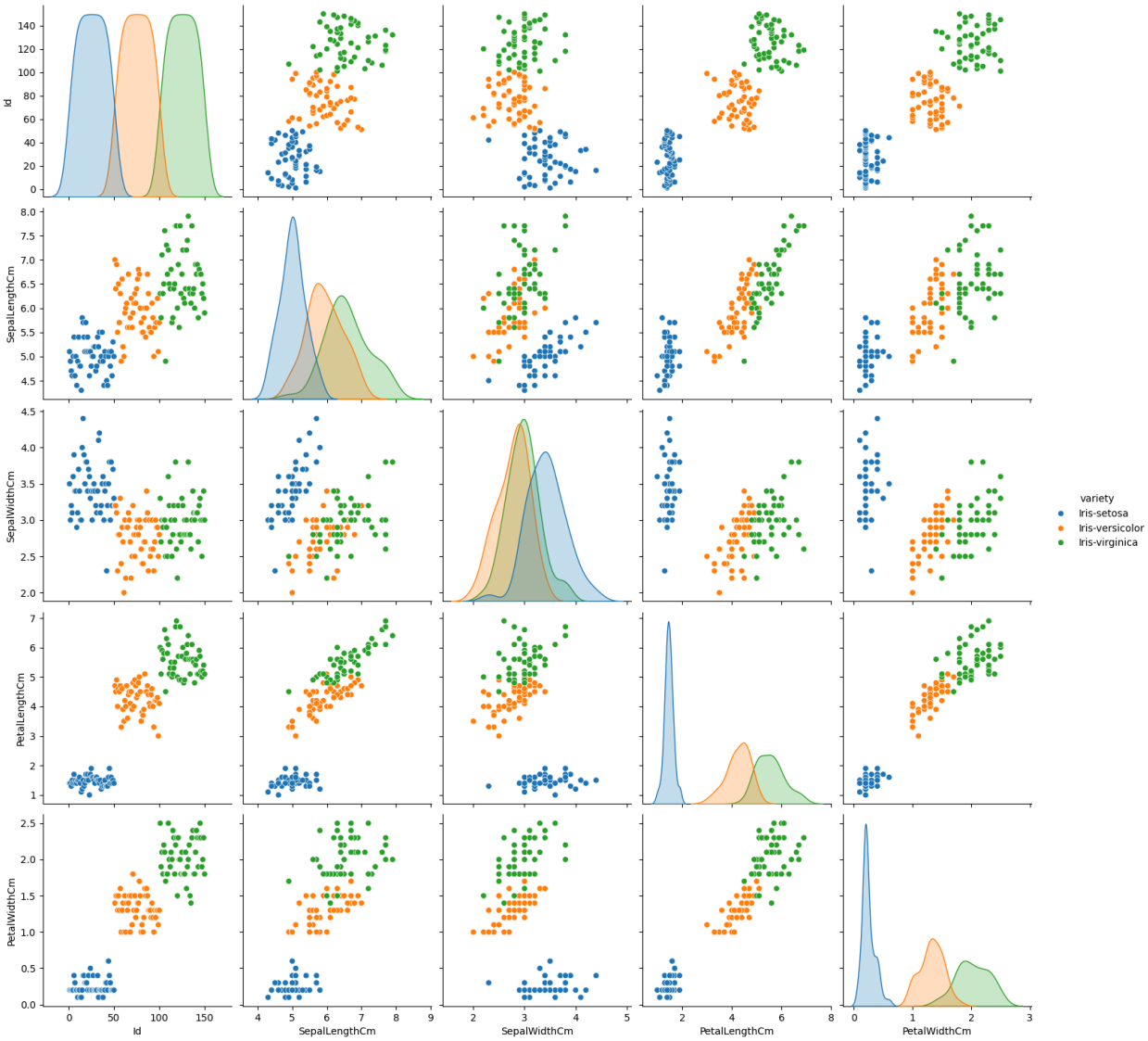
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   variety        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
               Id  SepalLengthCm  SepalWidthCm  PetalLengthCm
PetalWidthCm
count  150.000000     150.000000    150.000000     150.000000
150.000000
mean    75.500000       5.843333      3.057333       3.758000      1.199333
std     43.445368       0.828066      0.435866       1.765298      0.762238
min      1.000000       4.300000      2.000000       1.000000      0.100000
25%     38.250000       5.100000      2.800000       1.600000      0.300000
50%     75.500000       5.800000      3.000000       4.350000      1.300000
75%    112.750000       6.400000      3.300000       5.100000      1.800000
max    150.000000       7.900000      4.400000       6.900000      2.500000
variety
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

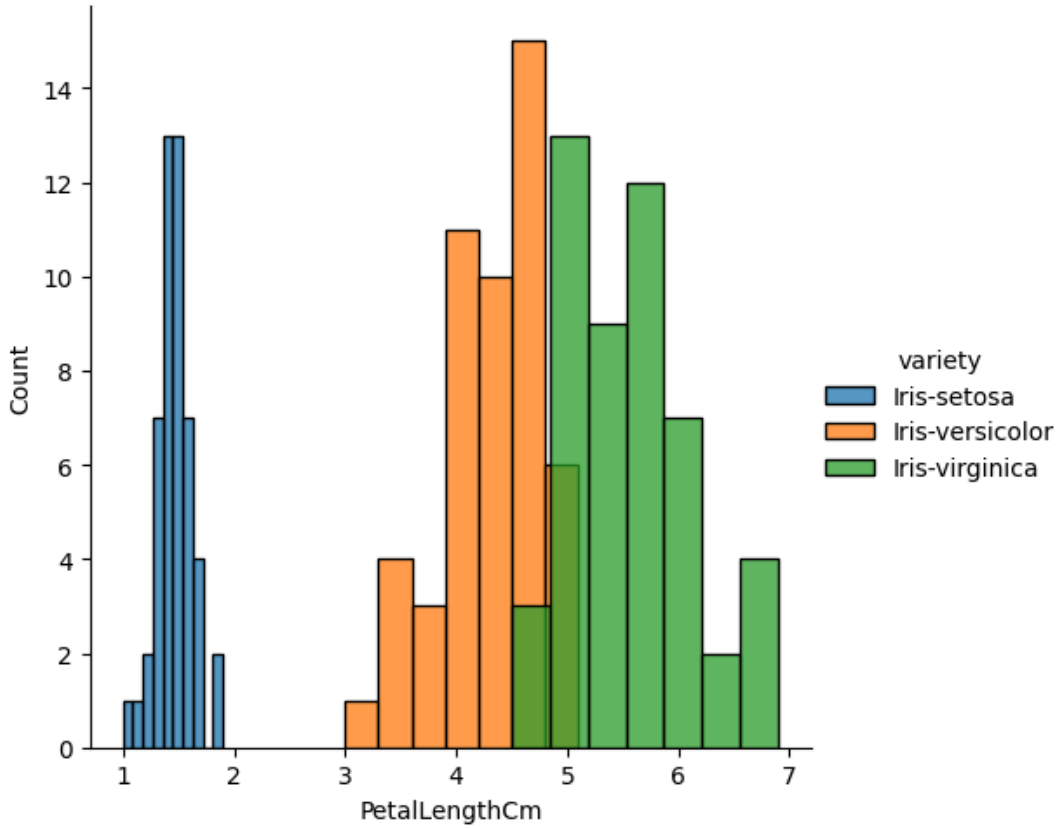


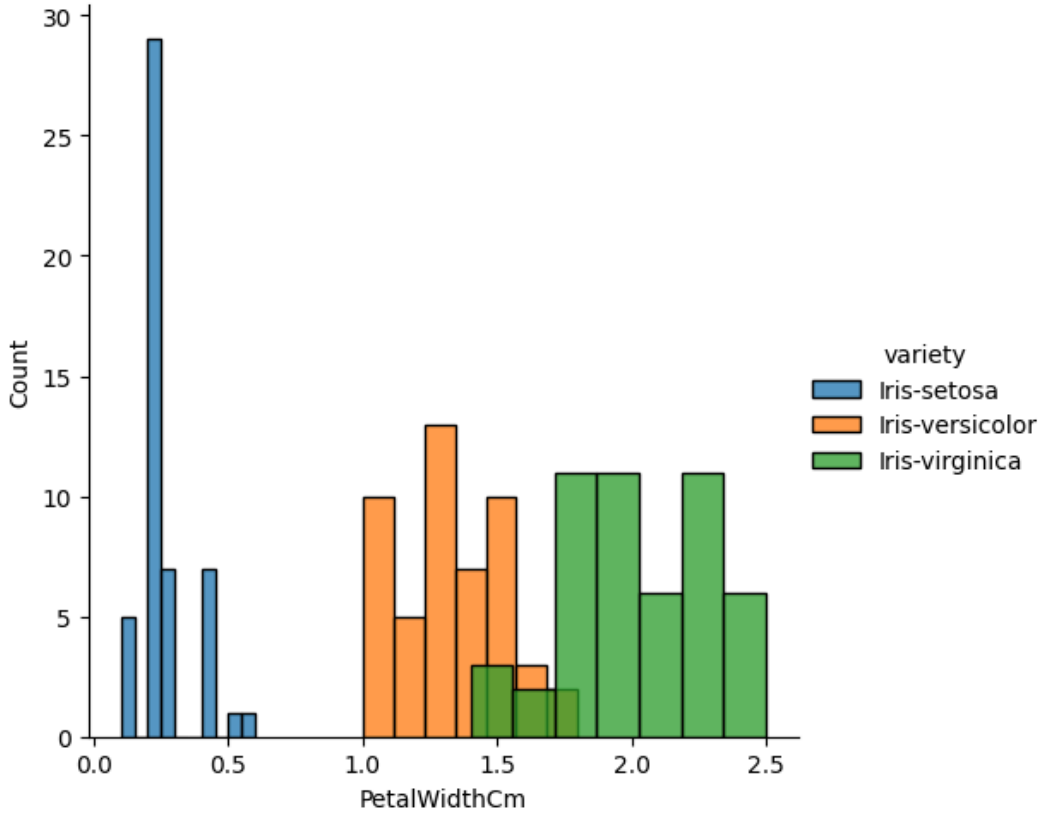
	Iris-setosa	Iris-versicolor	Iris-virginica	Id	SepalLengthCm	\
0	True	False	False	1	5.1	
1	True	False	False	2	4.9	
2	True	False	False	3	4.7	
3	True	False	False	4	4.6	
4	True	False	False	5	5.0	

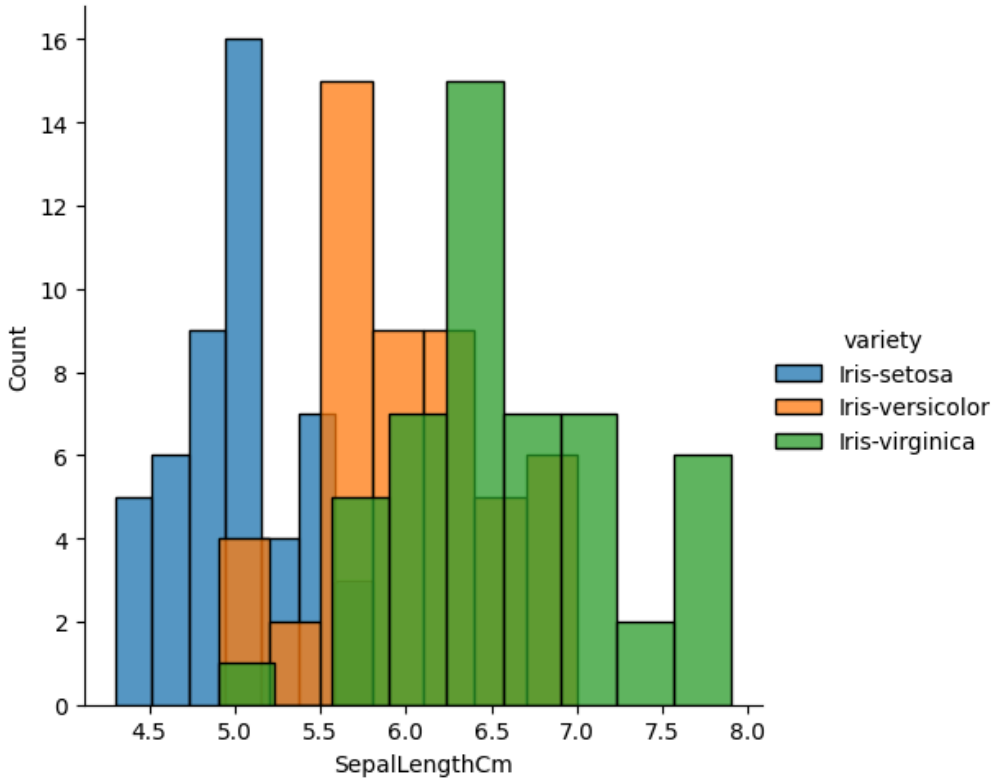
	SepalWidthCm	PetalLengthCm
0	3.5	1.4
1	3.0	1.4
2	3.2	1.3
3	3.1	1.5
4	3.6	1.4

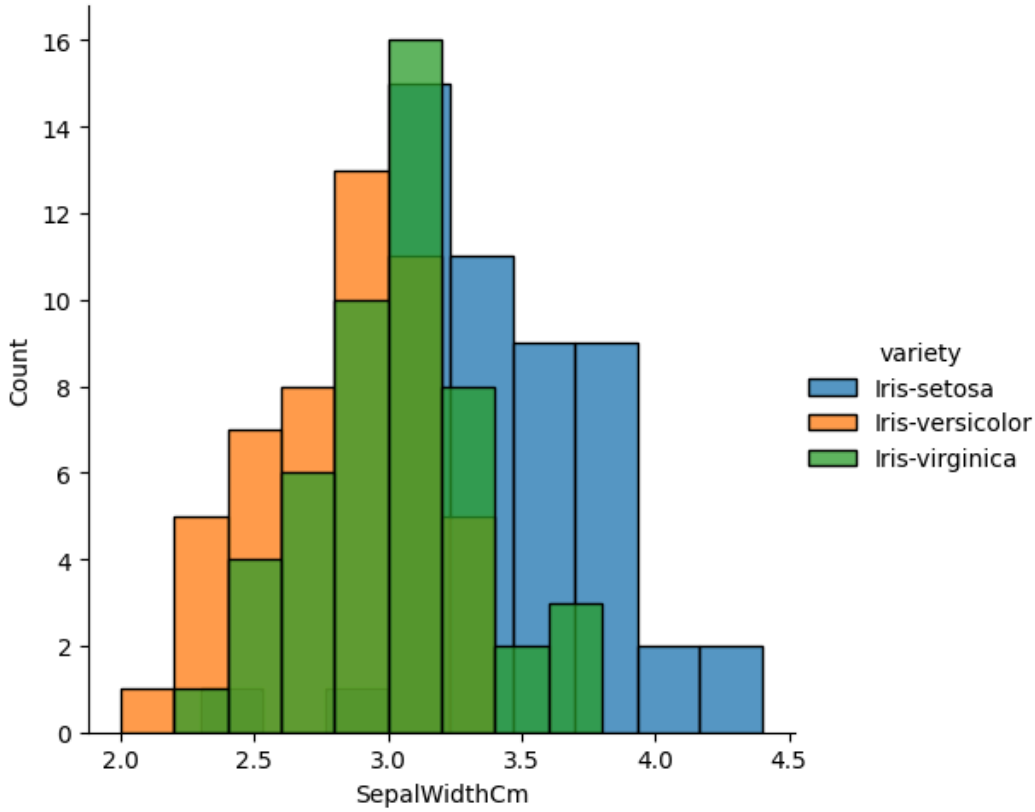

variety
Iris-setosa
Iris-versicolor
Iris-virginica
Iris-setosa
Iris-versicolor
Iris-virginica
SepalWidthCm
SepalLengthCm
0
1
2
3
4
1
2
3
4
5
6
7
8


Id
SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
SepalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
SepalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica

EXPERIMENT NO: 10

Experiment to understand array function in Data science.

Program:

```
import numpy as np
array=np.random.randint(1,100,9)
print(array)
print(np.sqrt(array))
print(array.ndim)
new_array=array.reshape(3,3)
print(new_array)
print(new_array.ndim)
print(new_array.ravel())
newm=new_array.reshape(3,3)
print(newm)
print(newm[2,1:3])
print(newm[1:2,1:3])
print(new_array[0:3,0:0])
print(new_array[0:2,0:1])
print(new_array[0:3,0:1])
print(new_array[1:3])
```

Output:

```
[21 46 93 65 35 87 19 74 22]
[4.58257569 6.78232998 9.64365076 8.06225775 5.91607978 9.32737905
 4.35889894 8.60232527 4.69041576]
1
[[21 46 93]
 [65 35 87]
 [19 74 22]]
2
[21 46 93 65 35 87 19 74 22]
[[21 46 93]
 [65 35 87]
 [19 74 22]]
```

```
[74 22]
[[35 87]]
[]
[[21]
 [65]]
[[21]
 [65]
 [19]]
[[65 35 87]
 [19 74 22]]
```

EXPERIMENT NO: 11

Experiment to understand pandas library use case in Data Science

Program:

```
import numpy as np
import pandas as pd
list=[[1,'Smith',50000],[2,'Jones',60000]]
df=pd.DataFrame(list)
print(df)
df.columns=['Empd','Name','Salary']
print(df)
print(df.info())
df=pd.read_csv("50_Startups.csv")
print(df.info())
print(df.head())
print(df.tail())
df=pd.read_csv("employee.csv")
print(df.head())
print(df.tail())
print(df.info())
print(df.salary)
print(type(df.salary))
print(df.salary.mean())
print(df.salary.median())
print(df.salary.mode())
print(df.salary.var())
print(df.salary.std())
print(df.describe())
print(df.describe(include='all'))
empCol=df.columns
print(empCol)
emparray=df.values
print(emparray)
employee_DF=pd.DataFrame(emparray,columns=empCol)
print(employee_DF)
```

Output:

```
  0      1      2
0  1  Smith  50000
1  2  Jones  60000
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
None
   R&D Spend  Administration  Marketing Spend       State     Profit
0   74308.32       180893.92         29300.28    New York   68422.31
1  157853.57       154642.93        304566.74    New York  166448.03
2  126139.12       176832.36        158031.97     Florida  123038.04
```

```
3  106805.48       170801.69        246399.66  California  123228.12
4   42622.70       130716.50        427942.75  California   88386.53
    R&D Spend  Administration  Marketing Spend       State     Profit
45  116065.73       116662.41         31783.56  California   94265.10
46   65198.11       120568.93        292351.77  California   93281.48
47   95409.86       107718.04        243718.96     Florida  111255.33
48   99272.99        53431.58         38422.83     Florida   80461.01
49   46803.90        64565.34        141784.14  California   58761.20
   emp_id                 name  salary
0       1      SREE VARSINI K S    5000
1       2           SREEMATHI B    6000
2       3              SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH H G    8000
   emp_id                 name  salary
0       1      SREE VARSINI K S    5000
1       2           SREEMATHI B    6000
2       3              SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH H G    8000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp_id  5 non-null      int64
 1   name    5 non-null      object
 2   salary  5 non-null      int64
dtypes: int64(2), object(1)
memory usage: 252.0+ bytes
None
0    5000
1    6000
2    7000
3    5000
```

```
4    8000
Name: salary, dtype: int64
<class 'pandas.core.series.Series'>
6200.0
6000.0
0    5000
Name: salary, dtype: int64
1700000.0
1303.8404810405298
         emp_id       salary
count  5.000000     5.000000
mean   3.000000  6200.000000
std    1.581139  1303.840481
min    1.000000  5000.000000
25%    2.000000  5000.000000
50%    3.000000  6000.000000
75%    4.000000  7000.000000
max    5.000000  8000.000000
         emp_id              name       salary
count   5.000000                 5     5.000000
unique       NaN                 5          NaN
top          NaN  SREE VARSINI K S          NaN
freq         NaN                 1          NaN
mean    3.000000               NaN  6200.000000
std     1.581139               NaN  1303.840481
min     1.000000               NaN  5000.000000
25%     2.000000               NaN  5000.000000
50%     3.000000               NaN  6000.000000
75%     4.000000               NaN  7000.000000
max     5.000000               NaN  8000.000000
Index(['emp_id', 'name', 'salary'], dtype='object')
[[1 'SREE VARSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
```

```
 [5 'SRI AKASH H G' 8000]]
```

	emp_id	name	salary
0	1	SREE VARSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH H G	8000

EXPERIMENT NO: 12

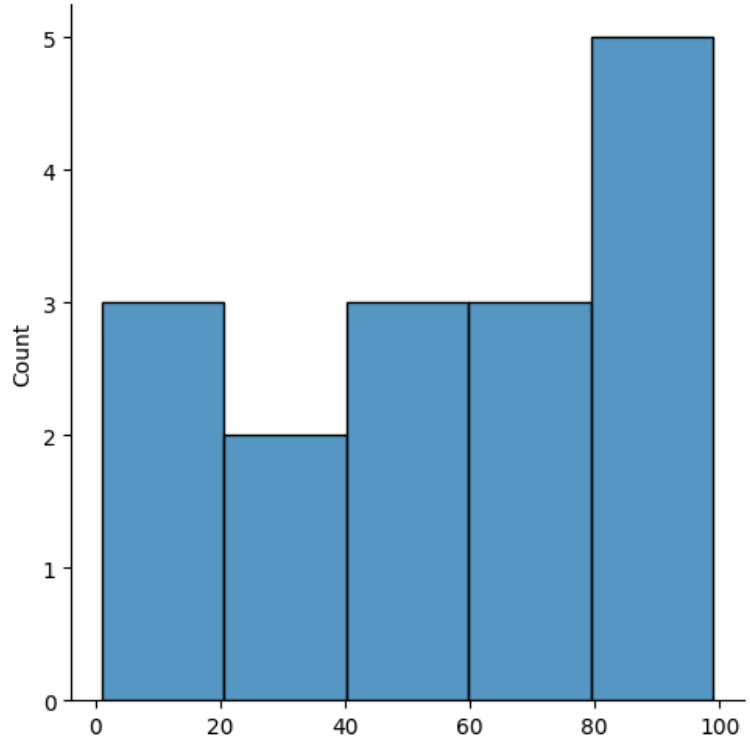
Experiment to detect outliers in a given dataset

Program:

```
import numpy as np
array = np.random.randint(1, 100, 16)
print(array)
print(array.mean())
print(np.percentile(array, 25))
print(np.percentile(array, 50))
print(np.percentile(array, 75))
print(np.percentile(array, 100))
def outDetection(array):
    sorted(array)
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur
lr, ur = outDetection(array)
print(lr, ur)
import seaborn as sns
%matplotlib inline
sns.displot(array)
sns.distplot(array)
new_array = array[(array > lr) & (array < ur)]
print(new_array)
sns.displot(new_array)
lr1, ur1 = outDetection(new_array)
print(lr1, ur1)
final_array = new_array[(new_array > lr1) & (new_array < ur1)]
print(final_array)
sns.distplot(final_array)
```

Output:
[99 57 37 75 24 95 78 82 69 90 55 89 1 16 12 46]
57.8125
33.75
63.0
83.75
99.0
-41.25 158.75

<seaborn.axisgrid.FacetGrid at 0x7fae16eacad0>



/tmp/ipykernel_2265/1133588802.py:1: UserWarning:

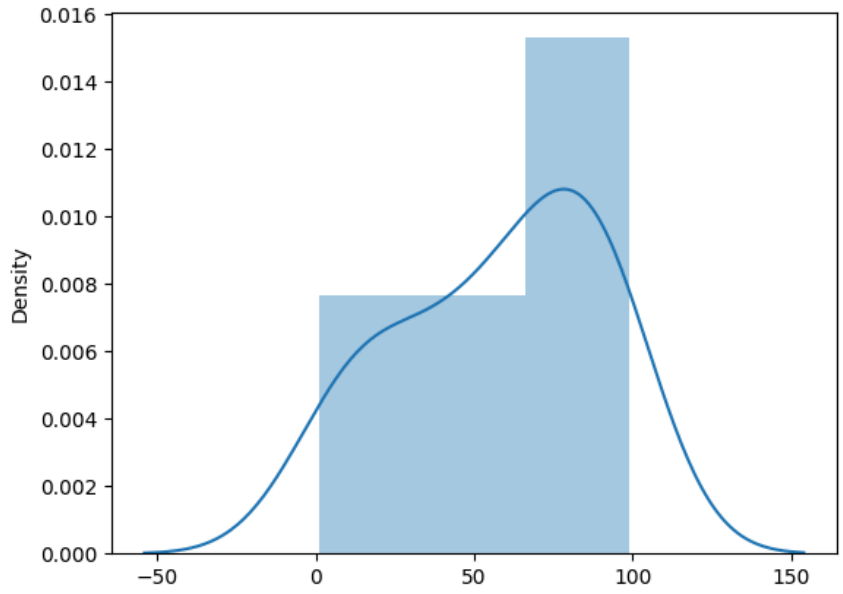
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

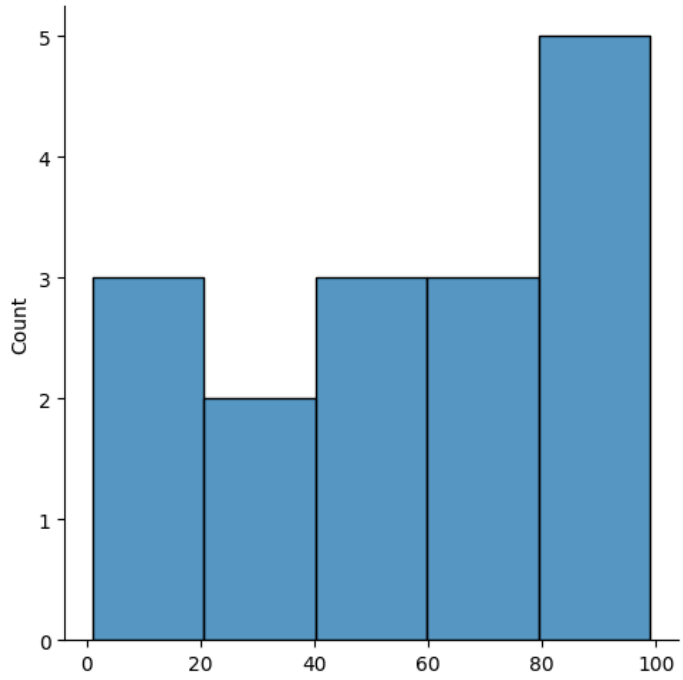
```
sns.distplot(array)
```

```
<Axes: ylabel='Density'>
```




```
[99 57 37 75 24 95 78 82 69 90 55 89  1 16 12 46]
```

```
<seaborn.axisgrid.FacetGrid at 0x7fae14ac3d90>
```



-41.25 158.75
[99 57 37 75 24 95 78 82 69 90 55 89 1 16 12 46]

/tmp/ipykernel_2265/2791332899.py:5: UserWarning:

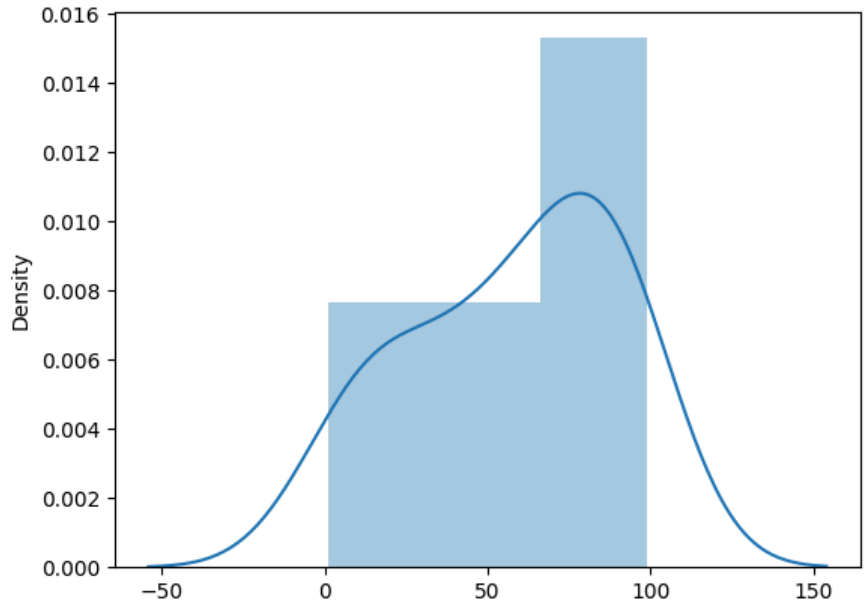
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

```
sns.distplot(final_array)
```

<Axes: ylabel='Density'>



EXPERIMENT NO: 13

Experiment to handle misssing values and inapproriate data in a given dataset

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv("Hotel_Dataset.csv")
print(df)
print(df.duplicated())
print(df.info())
df.drop_duplicates(inplace=True)
print(df)
print(len(df))
index=np.array(list(range(0,len(df))))
df.set_index(index,inplace=True)
print(index)
print(df)
df.drop(['Age_Group.1'],axis=1,inplace=True)
print(df)
df.CustomerID.loc[df.CustomerID<0]=np.nan
df.Bill.loc[df.Bill<0]=np.nan
df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
print(df)
df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan
print(df)
print(df.Age_Group.unique())
print(df.Hotel.unique())
df.Hotel.replace(['Ibys'],'Ibis',inplace=True)
print(df.FoodPreference.unique)
df.FoodPreference.replace(['Vegetarian','veg'],'Veg',inplace=True)
df.FoodPreference.replace(['non-Veg'],'Non-Veg',inplace=True)
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()),inplace=True)
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
df.Bill.fillna(round(df.Bill.mean()),inplace=True)
print(df)
```

Output:

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0           1     20-25           4       Ibis           veg  1300
1           2     30-35           5  LemonTree        Non-Veg  2000
2           3     25-30           6     RedFox           Veg  1322
3           4     20-25          -1  LemonTree           Veg  1234
4           5       35+           3       Ibis     Vegetarian   989
5           6       35+           3       Ibys        Non-Veg  1909
6           7       35+           4     RedFox     Vegetarian  1000
7           8     20-25           7  LemonTree           Veg  2999
8           9     25-30           2       Ibis        Non-Veg  3456
9           9     25-30           2       Ibis        Non-Veg  3456
10         10     30-35           5     RedFox        non-Veg -6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2           40000       20-25
1         3           59000       30-35
2         2           30000       25-30
3         2          120000       20-25
4         2           45000         35+
5         2          122220         35+
6        -1           21122         35+
7       -10          345673       20-25
8         3          -99999       25-30
9         3          -99999       25-30
10        4           87777       30-35
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9     True
10   False
dtype: bool
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
```

```
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes
None
    CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0            1     20-25            4       Ibis           veg  1300
1            2     30-35            5  LemonTree        Non-Veg  2000
2            3     25-30            6     RedFox           Veg  1322
3            4     20-25           -1  LemonTree           Veg  1234
4            5       35+            3       Ibis     Vegetarian   989
5            6       35+            3       Ibys        Non-Veg  1909
6            7       35+            4     RedFox     Vegetarian  1000
7            8     20-25            7  LemonTree           Veg  2999
8            9     25-30            2       Ibis        Non-Veg  3456
10          10     30-35            5     RedFox        non-Veg -6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2           40000       20-25
1         3           59000       30-35
2         2           30000       25-30
3         2          120000       20-25
4         2           45000         35+
5         2          122220         35+
6        -1           21122         35+
7       -10          345673       20-25
8         3          -99999       25-30
10        4           87777       30-35
10
[0 1 2 3 4 5 6 7 8 9]
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  NoOfPax  \
```

0	1	20-25	4	Ibis	veg	1300	2
1	2	30-35	5	LemonTree	Non-Veg	2000	3
2	3	25-30	6	RedFox	Veg	1322	2
3	4	20-25	-1	LemonTree	Veg	1234	2
4	5	35+	3	Ibis	Vegetarian	989	2
5	6	35+	3	Ibys	Non-Veg	1909	2
6	7	35+	4	RedFox	Vegetarian	1000	-1
7	8	20-25	7	LemonTree	Veg	2999	-10
8	9	25-30	2	Ibis	Non-Veg	3456	3
9	10	30-35	5	RedFox	non-Veg	-6755	4

	EstimatedSalary	Age_Group.1
0	40000	20-25
1	59000	30-35
2	30000	25-30
3	120000	20-25
4	45000	35+
5	122220	35+
6	21122	35+
7	345673	20-25
8	-99999	25-30
9	87777	30-35

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	\
0	1	20-25	4	Ibis	veg	1300	2	
1	2	30-35	5	LemonTree	Non-Veg	2000	3	
2	3	25-30	6	RedFox	Veg	1322	2	
3	4	20-25	-1	LemonTree	Veg	1234	2	
4	5	35+	3	Ibis	Vegetarian	989	2	
5	6	35+	3	Ibys	Non-Veg	1909	2	
6	7	35+	4	RedFox	Vegetarian	1000	-1	
7	8	20-25	7	LemonTree	Veg	2999	-10	
8	9	25-30	2	Ibis	Non-Veg	3456	3	
9	10	30-35	5	RedFox	non-Veg	-6755	4	

	EstimatedSalary
0	40000
1	59000
2	30000
3	120000
4	45000
5	122220

```
6          21122
7         345673
8         -99999
9          87777
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25           4       Ibis           veg  1300.0
1         2.0     30-35           5  LemonTree        Non-Veg  2000.0
2         3.0     25-30           6     RedFox           Veg  1322.0
3         4.0     20-25          -1  LemonTree           Veg  1234.0
4         5.0       35+           3       Ibis     Vegetarian   989.0
5         6.0       35+           3       Ibys        Non-Veg  1909.0
6         7.0       35+           4     RedFox     Vegetarian  1000.0
7         8.0     20-25           7  LemonTree           Veg  2999.0
8         9.0     25-30           2       Ibis        Non-Veg  3456.0
9        10.0     30-35           5     RedFox        non-Veg     NaN

   NoOfPax  EstimatedSalary
0        2          40000.0
1        3          59000.0
2        2          30000.0
3        2         120000.0
4        2          45000.0
5        2         122220.0
6       -1          21122.0
7      -10         345673.0
8        3             NaN
9        4          87777.0
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25           4       Ibis           veg  1300.0
1         2.0     30-35           5  LemonTree        Non-Veg  2000.0
2         3.0     25-30           6     RedFox           Veg  1322.0
3         4.0     20-25          -1  LemonTree           Veg  1234.0
4         5.0       35+           3       Ibis     Vegetarian   989.0
5         6.0       35+           3       Ibys        Non-Veg  1909.0
6         7.0       35+           4     RedFox     Vegetarian  1000.0
7         8.0     20-25           7  LemonTree           Veg  2999.0
8         9.0     25-30           2       Ibis        Non-Veg  3456.0
9        10.0     30-35           5     RedFox        non-Veg     NaN

   NoOfPax  EstimatedSalary
0      2.0          40000.0
```

```
1     3.0         59000.0
2     2.0         30000.0
3     2.0        120000.0
4     2.0         45000.0
5     2.0        122220.0
6     NaN         21122.0
7     NaN        345673.0
8     3.0             NaN
9     4.0         87777.0
['20-25' '30-35' '25-30' '35+']
['Ibis' 'LemonTree' 'RedFox' 'Ibys']
<bound method Series.unique of 0           veg
1       Non-Veg
2           Veg
3           Veg
4     Vegetarian
5       Non-Veg
6     Vegetarian
7           Veg
8       Non-Veg
9       non-Veg
Name: FoodPreference, dtype: object>
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            Veg  1300.0
1         2.0     30-35            5  LemonTree       Non-Veg  2000.0
2         3.0     25-30            6     RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis            Veg   989.0
5         6.0       35+            3       Ibis       Non-Veg  1909.0
6         7.0       35+            4     RedFox            Veg  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis       Non-Veg  3456.0
9        10.0     30-35            5     RedFox       Non-Veg  1801.0

   NoOfPax  EstimatedSalary
0      2.0         40000.0
1      3.0         59000.0
2      2.0         30000.0
3      2.0        120000.0
4      2.0         45000.0
5      2.0        122220.0
```

6 2.0 21122.0
7 2.0 345673.0
8 3.0 96755.0
9 4.0 87777.0

/tmp/ipykernel_928/3883048516.py:16: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

df.CustomerID.loc[df.CustomerID<0]=np.nan
/tmp/ipykernel_928/3883048516.py:16: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df.CustomerID.loc[df.CustomerID<0]=np.nan
/tmp/ipykernel_928/3883048516.py:17: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
  df.Bill.loc[df.Bill<0]=np.nan
/tmp/ipykernel_928/3883048516.py:17: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df.Bill.loc[df.Bill<0]=np.nan
/tmp/ipykernel_928/3883048516.py:18: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

  df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
/tmp/ipykernel_928/3883048516.py:18: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
/tmp/ipykernel_928/3883048516.py:20: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a DataFrame, like:
```

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
  df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan
```
```
/tmp/ipykernel_928/3883048516.py:20: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```
  df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan
```
```
/tmp/ipykernel_928/3883048516.py:24: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
```

```
  df.Hotel.replace(['Ibys'],'Ibis',inplace=True)
```
```
/tmp/ipykernel_928/3883048516.py:26: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
```

```
  df.FoodPreference.replace(['Vegetarian','veg'],'Veg',inplace=True)
```
```
/tmp/ipykernel_928/3883048516.py:30: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
```

EXPERIMENT NO: 14

Experiment to understand feature scaling

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('pre-process_datasample.csv')
print(df)
print(df.head())
df.Country.fillna(df.Country.mode()[0],inplace=True)
features=df.iloc[:,:-1].values
print(features)
label=df.iloc[:,-1].values
from sklearn.impute import SimpleImputer
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
age.fit(features[:,[1]])
Salary.fit(features[:,[2]])
SimpleImputer()
features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
print(features)
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
print(Country)
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
print(final_set)
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
print(feat_standard_scaler)
from sklearn.preprocessing import MinMaxScaler
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
print(feat_minmax_scaler)
```

Output:

```
  Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
  Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 nan]
 ['France' 35.0 58000.0]
 ['Spain' nan 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 63777.77777777778]
 ['France' 35.0 58000.0]
```

```
 ['Spain' 38.77777777777778 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
[[1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [1. 0. 0.]]
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]
[[ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  7.58874362e-01
   7.49473254e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.71150388e+00
  -1.43817841e+00]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01 -1.27555478e+00
  -8.91265492e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.13023841e-01
  -2.53200424e-01]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.77608893e-01
   6.63219199e-16]
```

```
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -5.48972942e-01
  -5.26656882e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00  0.00000000e+00
  -1.07356980e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  1.34013983e+00
   1.38753832e+00]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.63077256e+00
   1.75214693e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -2.58340208e-01
   2.93712492e-01]]
[[1.         0.         0.         0.73913043 0.68571429]
 [0.         0.         1.         0.         0.        ]
 [0.         1.         0.         0.13043478 0.17142857]
 [0.         0.         1.         0.47826087 0.37142857]
 [0.         1.         0.         0.56521739 0.45079365]
 [1.         0.         0.         0.34782609 0.28571429]
 [0.         0.         1.         0.51207729 0.11428571]
 [1.         0.         0.         0.91304348 0.88571429]
 [0.         1.         0.         1.         1.        ]
 [1.         0.         0.         0.43478261 0.54285714]]
```

/tmp/ipykernel_647/1037654059.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df.Country.fillna(df.Country.mode()[0],inplace=True)
```

EXPERIMENT No: 15

Experiment to understand the data preprocessing in data science

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv("pre-process_datasample.csv")
print(df)
print(df.info())
print(df.Country.mode())
print(df.Country.mode()[0])
print(type(df.Country.mode()))
df.Country.fillna(df.Country.mode()[0],inplace=True)
df.Age.fillna(df.Age.median(),inplace=True)
df.Salary.fillna(round(df.Salary.mean()),inplace=True)
print(df)
print(pd.get_dummies(df.Country))
updated_dataset=pd.concat([pd.get_dummies(df.Country),df.iloc[:,[1,2,3]]],
axis=1)
print(updated_dataset)
print(df.info())
updated_dataset.Purchased.replace(['No','Yes'],[0,1],inplace=True)
print(updated_dataset)
```

Output:

```
  Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
```

```
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
None
0    France
Name: Country, dtype: object
France
<class 'pandas.core.series.Series'>
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
   France  Germany  Spain
0    True    False  False
1   False    False   True
2   False     True  False
3   False    False   True
4   False     True  False
5    True    False  False
6   False    False   True
7    True    False  False
8   False     True  False
9    True    False  False
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1   False    False   True  27.0  48000.0       Yes
2   False     True  False  30.0  54000.0        No
3   False    False   True  38.0  61000.0        No
4   False     True  False  40.0  63778.0       Yes
5    True    False  False  35.0  58000.0       Yes
6   False    False   True  38.0  52000.0        No
7    True    False  False  48.0  79000.0       Yes
```

```
8   False     True  False  50.0  83000.0        No
9    True    False  False  37.0  67000.0       Yes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        10 non-null     float64
 2   Salary     10 non-null     float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
None
   France  Germany  Spain   Age   Salary  Purchased
0    True    False  False  44.0  72000.0          0
1   False    False   True  27.0  48000.0          1
2   False     True  False  30.0  54000.0          0
3   False    False   True  38.0  61000.0          0
4   False     True  False  40.0  63778.0          1
5    True    False  False  35.0  58000.0          1
6   False    False   True  38.0  52000.0          0
7    True    False  False  48.0  79000.0          1
8   False     True  False  50.0  83000.0          0
9    True    False  False  37.0  67000.0          1

/tmp/ipykernel_2456/692133200.py:9: FutureWarning: A value is trying to be set on a copy of a
DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the
intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value},
inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the
original object.


  df.Country.fillna(df.Country.mode()[0],inplace=True)
/tmp/ipykernel_2456/692133200.py:10: FutureWarning: A value is trying to be set on a copy of a
DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the
intermediate object on which we are setting values always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
  df.Age.fillna(df.Age.median(),inplace=True)
```

```
/tmp/ipykernel_2456/692133200.py:11: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
```

```
  df.Salary.fillna(round(df.Salary.mean()),inplace=True)
```

```
/tmp/ipykernel_2456/692133200.py:17: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
```

```
  updated_dataset.Purchased.replace(['No','Yes'],[0,1],inplace=True)
```

```
/tmp/ipykernel_2456/692133200.py:17: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  updated_dataset.Purchased.replace(['No','Yes'],[0,1],inplace=True)
```

EXPERIMENT NO: 16

Experiment to understand EDA - Quantitative and Qualitative analysis

Program:

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips=sns.load_dataset('tips')
print(tips.head())
sns.displot(tips.total_bill,kde=True)
sns.displot(tips.total_bill,kde=False)
sns.jointplot(x=tips.tip,y=tips.total_bill)
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="reg")
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")
sns.pairplot(tips)
print(tips.time.value_counts())
sns.pairplot(tips,hue='time')
sns.pairplot(tips, hue='day')
plt.show()
plt.close()
sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()
plt.close()plt.figure()
sns.boxplot(y=tips.total_bill)
plt.show()
plt.close()
plt.figure()
sns.boxplot(y=tips.tip)
plt.show()
plt.close()

plt.figure()
sns.countplot(x=tips.day)
plt.show()
plt.close()
```

```
plt.figure()
sns.countplot(x=tips.sex)
plt.show()
plt.close()

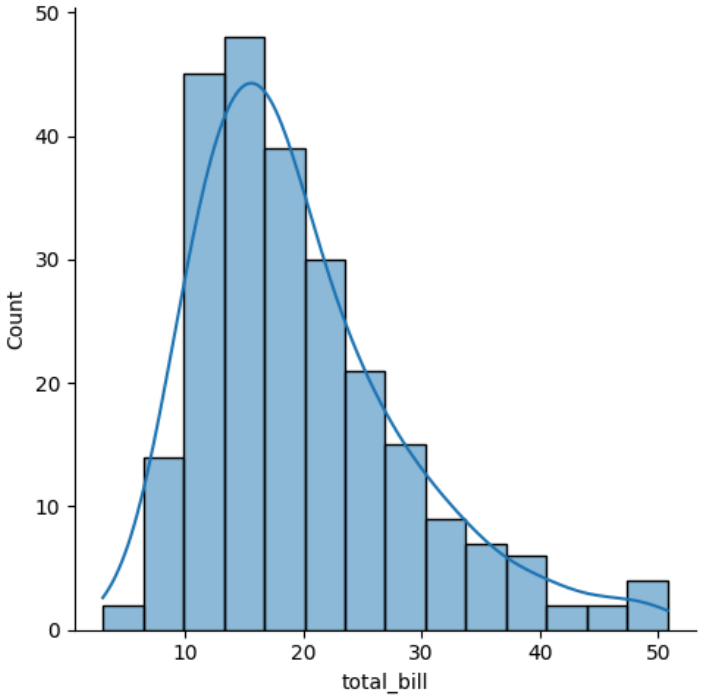
plt.figure()
tips.sex.value_counts().plot(kind='pie')
plt.show()
plt.close()

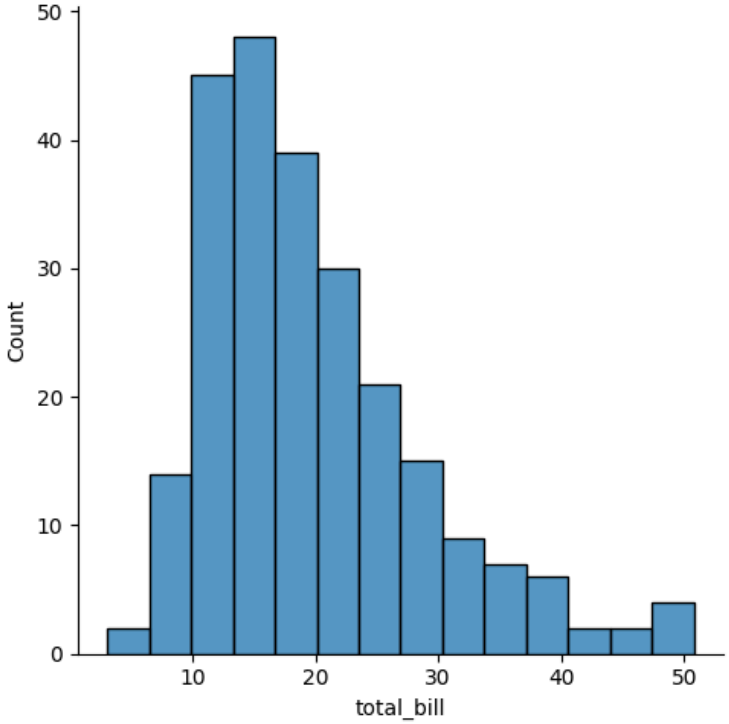
plt.figure()
tips.sex.value_counts().plot(kind='bar')
plt.show()
plt.close()

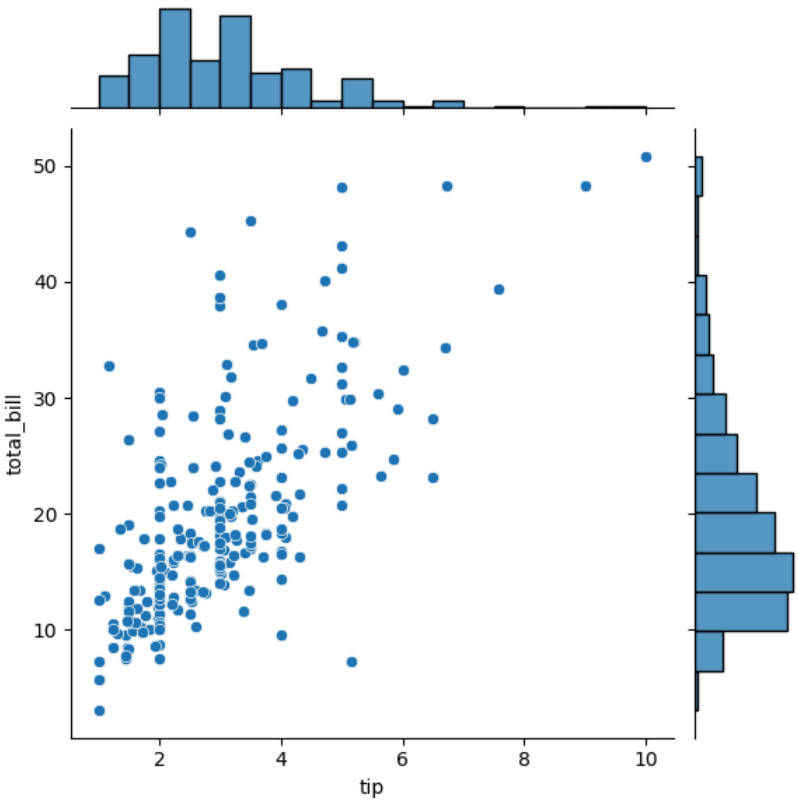
plt.figure()
sns.countplot(x=tips[tips.time == 'Dinner']['day'])
plt.show()
plt.close()
```

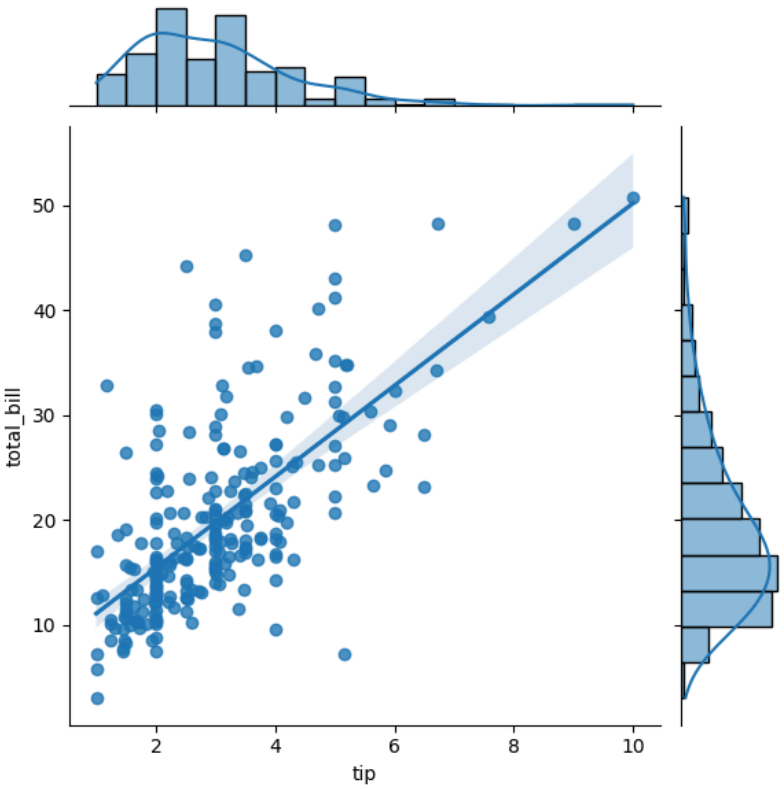
Output:

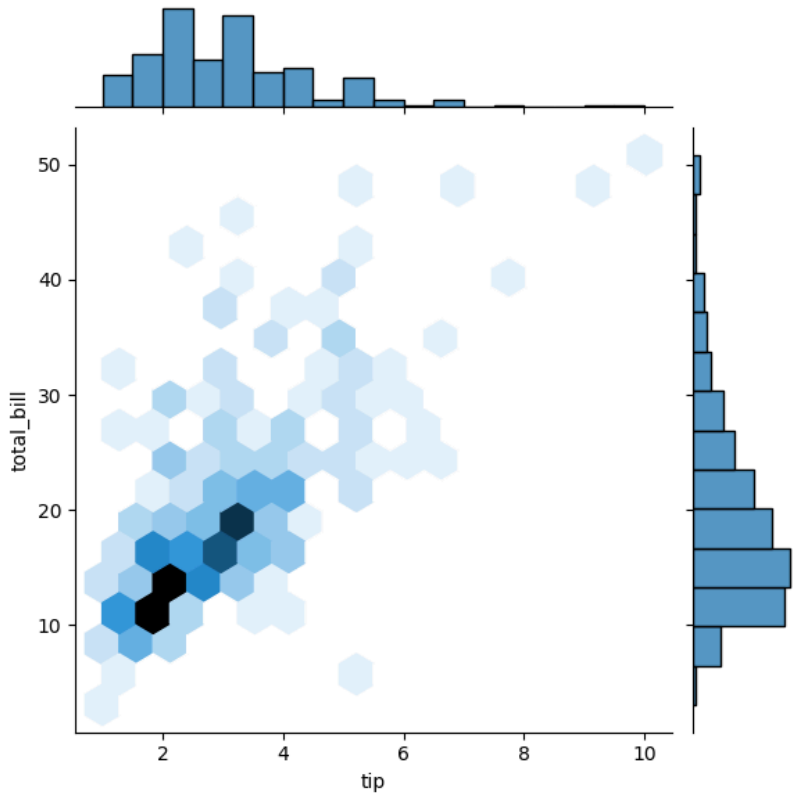
```
  total_bill   tip     sex smoker  day    time  size
0      16.99  1.01  Female     No  Sun  Dinner     2
1      10.34  1.66    Male     No  Sun  Dinner     3
2      21.01  3.50    Male     No  Sun  Dinner     3
3      23.68  3.31    Male     No  Sun  Dinner     2
4      24.59  3.61  Female     No  Sun  Dinner     4
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

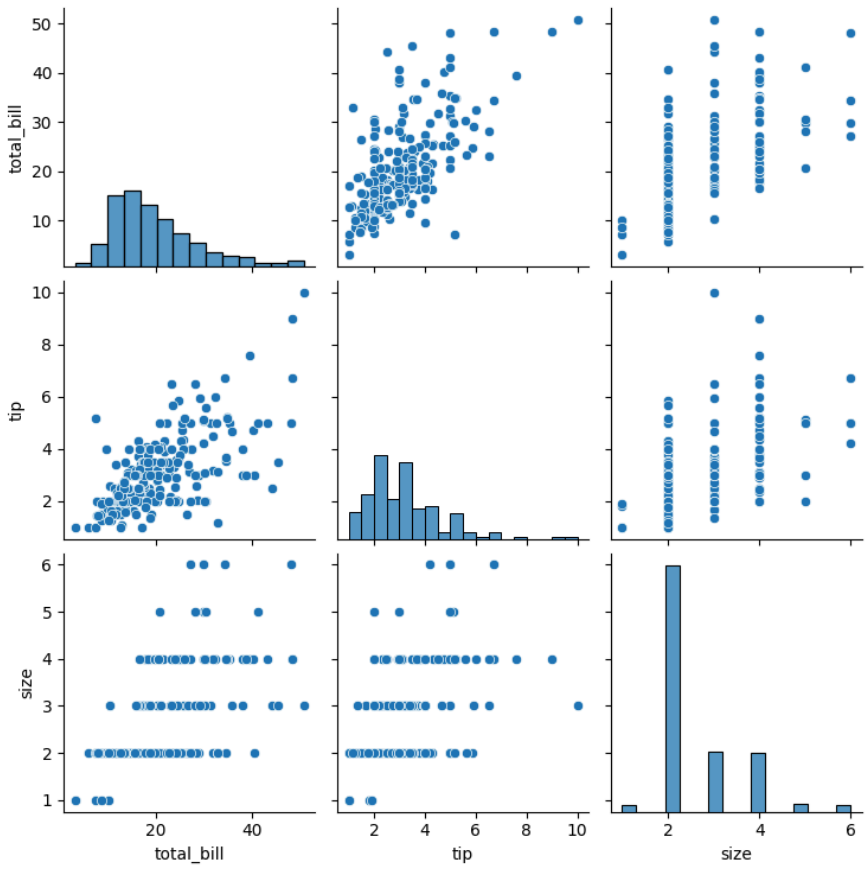

Count
total_bill
0
10
20
30
40
50

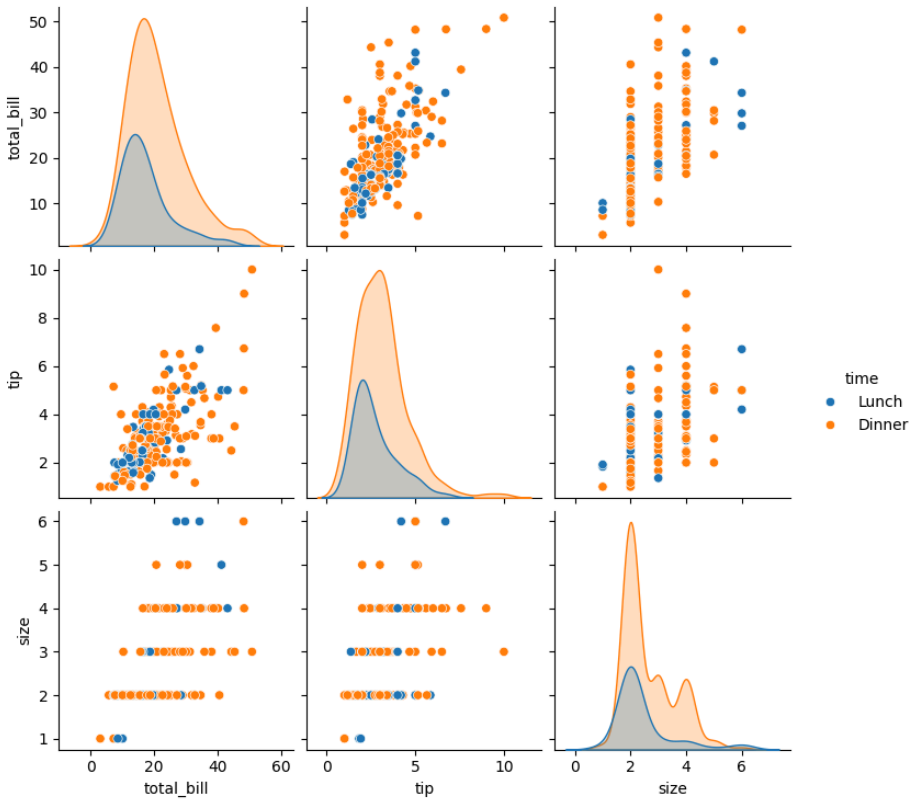

Count
total_bill
0
10
20
30
40
50

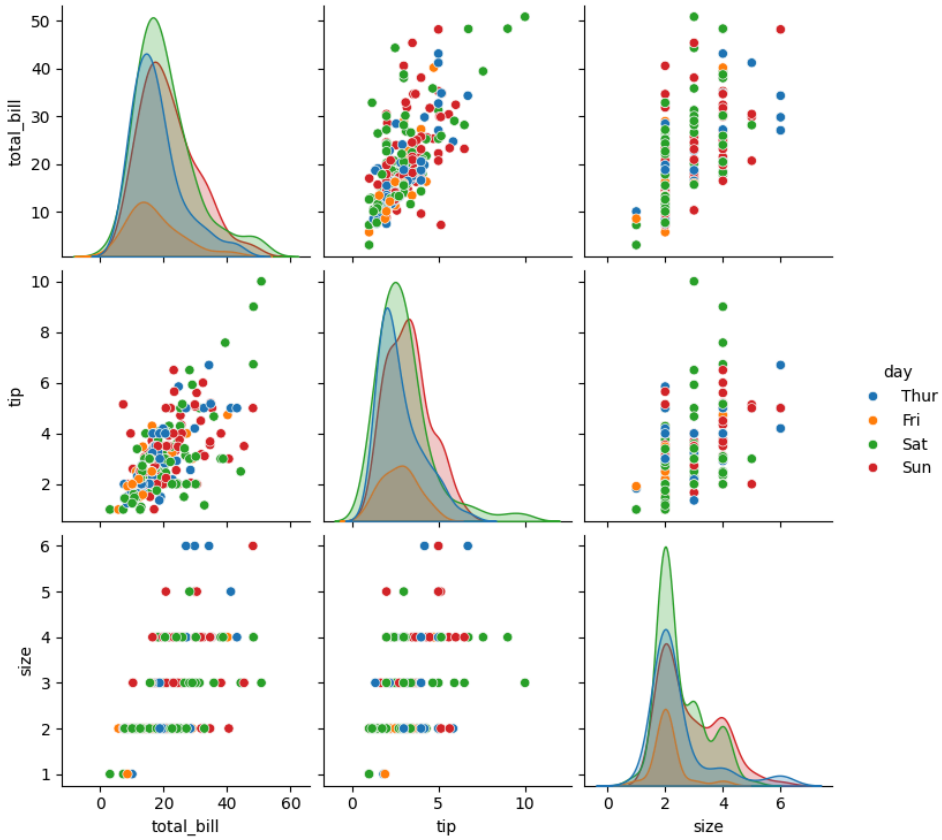

total_bill
tip
50
40
30
20
10
2
4
6
8
10

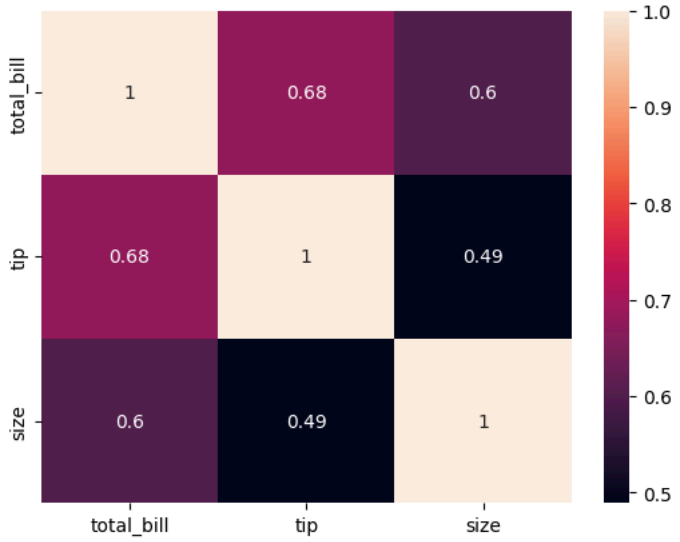

total_bill
tip
50
40
30
20
10
2
4
6
8
10

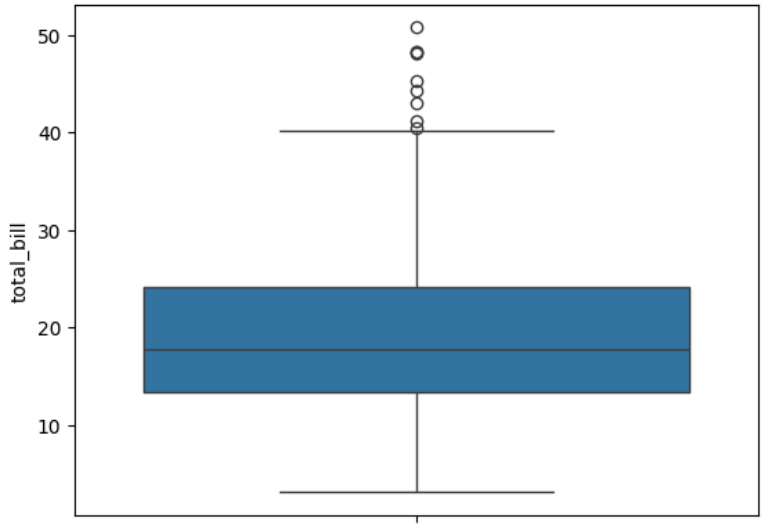

total_bill
tip
50
40
30
20
10
2
4
6
8
10

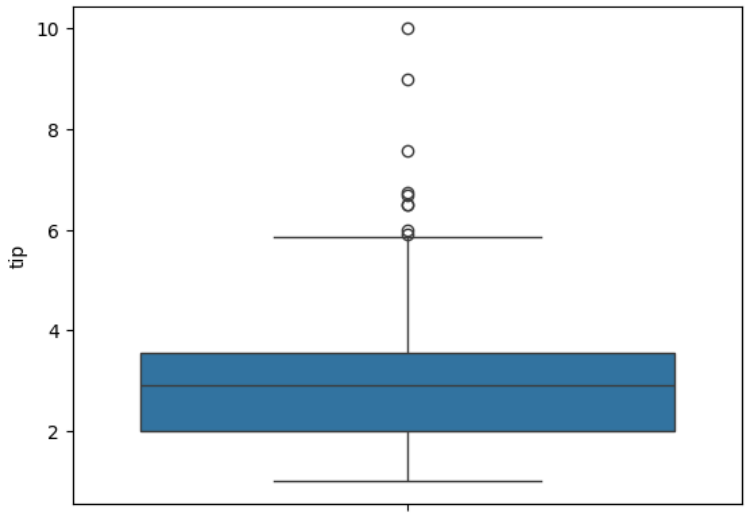

total_bill
tip
size
50
40
30
20
10
10
8
6
4
2
6
5
4
3
2
1
20
40
2
4
6
8
10
2
4
6

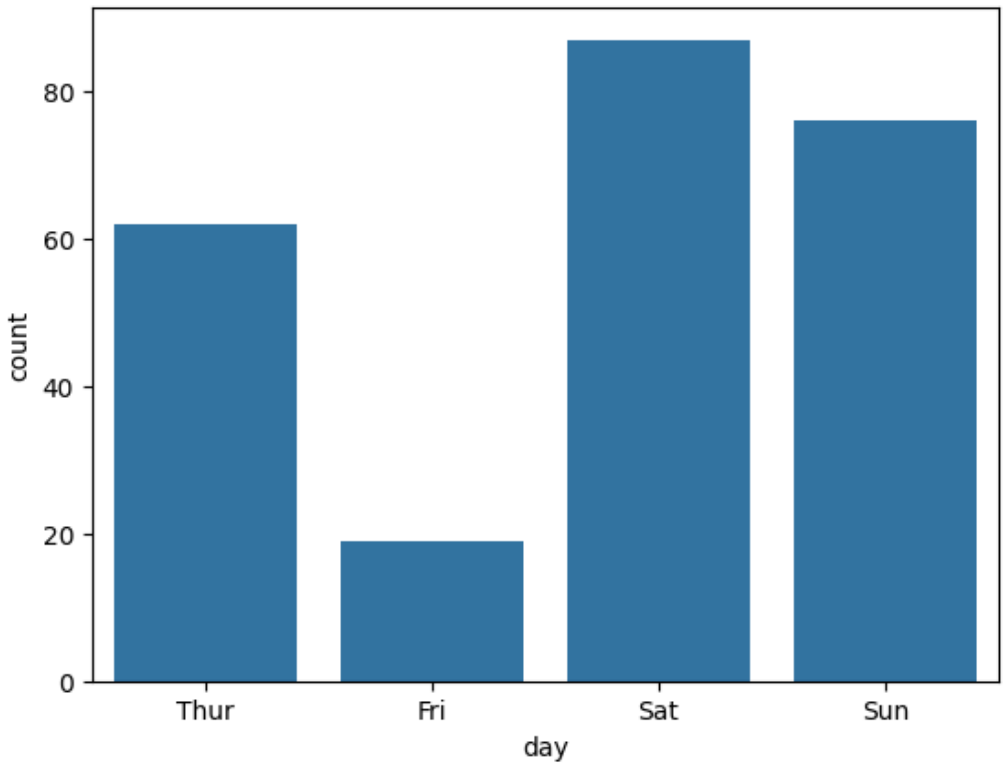

total_bill
tip
size
time
Lunch
Dinner

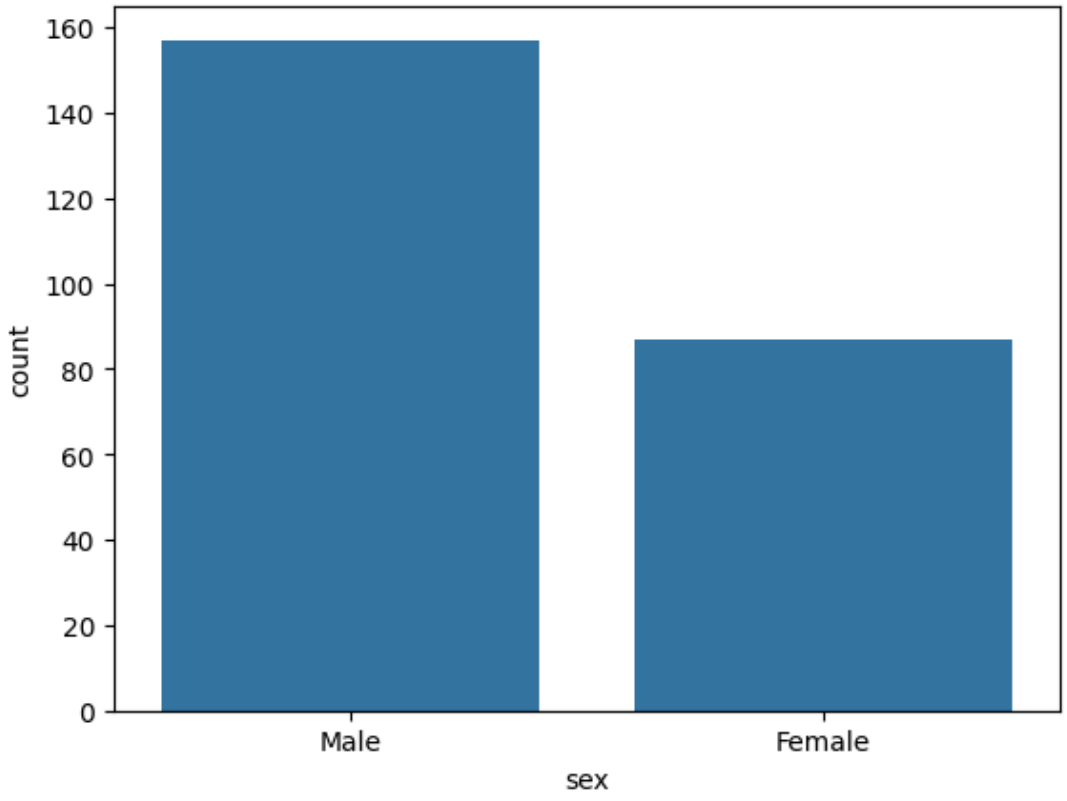

total_bill
tip
size
day
Thur
Fri
Sat
Sun

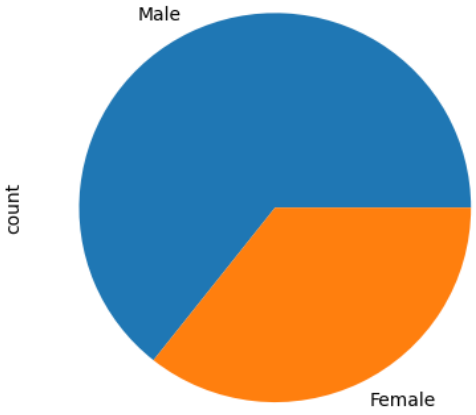

total_bill
tip
size
1
0.68
0.6
0.68
1
0.49
0.6
0.49
1
total_bill
tip
size
1.0
0.9
0.8
0.7
0.6
0.5

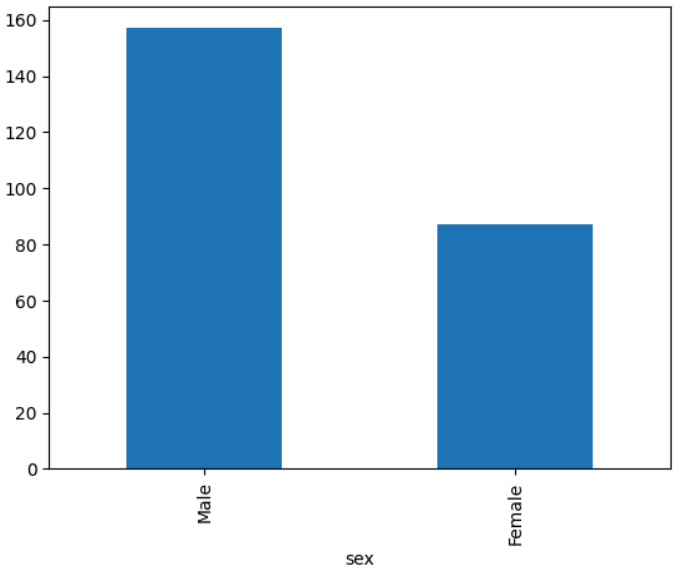

total_bill
50
40
30
20
10

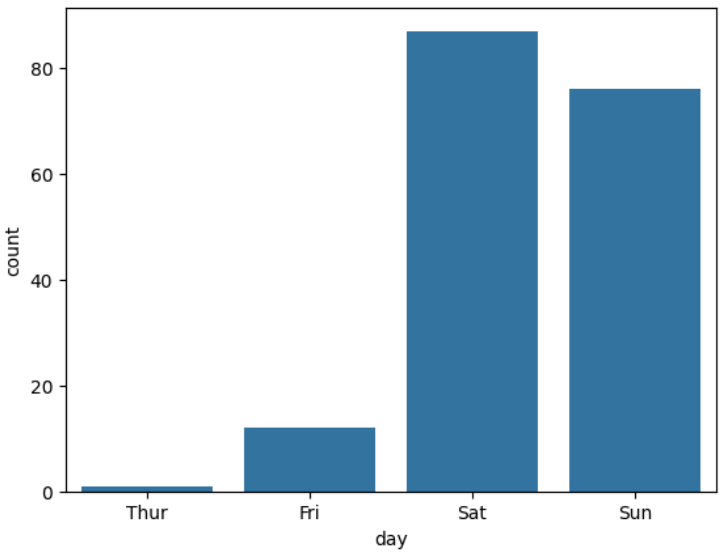

tip
10
8
6
4
2


80
60
40
20
0
count
Thur
Fri
Sat
Sun
day


160
140
120
100
80
60
40
20
0
count
Male
Female
sex


Male
count
Female


160
140
120
100
80
60
40
20
0
Male
Female
sex


80
60
40
20
0
count
Thur
Fri
Sat
Sun
day

EXPERIMENT No: 17

Experiment to understand Linear Regression for a given data set.

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
print(df.info())
df.dropna(inplace=True)
print(df.info())
print(df.describe())
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features, label,
test_size=0.2,random_state=42)
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
print(model.score(x_train,y_train))
print(model.score(x_test,y_test))
print(model.coef_)
print(model.intercept_)
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
model=pickle.load(open('SalaryPred.model','rb'))
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}:
".format(yr_of_exp,Salary))
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
None
       YearsExperience         Salary
count        30.000000      30.000000
mean          5.313333   76003.000000
std           2.837888   27414.429785
min           1.100000   37731.000000
25%           3.200000   56720.750000
50%           4.700000   65237.000000
75%           7.700000  100544.750000
max          10.500000  122391.000000
0.9645401573418146
0.9024461774180497
[[9423.81532303]]
[25321.58301178]
Enter Years of Experience: 44
Estimated Salary for 44.0 years of experience is [[439969.45722514]]:
```

EXPERIMENT NO: 18

Experiment to understand KNN algorithm for a given dataset

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Iris.csv')
print(df.info())
print(df.variety.value_counts())
print(df.head())
features=df.iloc[:,:-1].values
label=df.iloc[:,4].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,ran
dom_state=42)
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
from sklearn.metrics import confusion_matrix
print(confusion_matrix(label,model_KNN.predict(features)))
from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
```

```
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
   sepal.length  sepal.width  petal.length  petal.width variety
0           5.1          3.5           1.4          0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa
0.9666666666666667
1.0
[[50  0  0]
 [ 0 47  3]
 [ 0  1 49]]
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.98      0.94      0.96        50
   Virginica       0.94      0.98      0.96        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

EXPERIMENT NO: 19

Experiment to understand Logistic Regression for a given dataset.

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
print(df)
print(df.head())
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
print(features)
print(label)
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
for i in range(1,401):

x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.
2,random_state=42)
  model=LogisticRegression()
  model.fit(x_train,y_train)
  train_score=model.score(x_train,y_train)
  test_score=model.score(x_test,y_test)
  if test_score>train_score:
      print("Test {} Train {} Random State
{}".format(test_score,train_score,i))
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.
2,random_state=42)
finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)
print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))
from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
```

Output:

```
      User ID  Gender  Age  EstimatedSalary  Purchased
0    15624510    Male   19            19000          0
1    15810944    Male   35            20000          0
2    15668575  Female   26            43000          0
3    15603246  Female   27            57000          0
4    15804002    Male   19            76000          0
..        ...     ...  ...              ...        ...
395  15691863  Female   46            41000          1
396  15706071    Male   51            23000          1
397  15654296  Female   50            20000          1
398  15755018    Male   36            33000          0
399  15594041  Female   49            36000          1

[400 rows x 5 columns]
   User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0
[[    19  19000]
 [    35  20000]
 [    26  43000]
 [    27  57000]
 [    19  76000]
 [    45 101000]
 [    39  59000]
 [    46  72000]
 [    28  82000]
 [    36  46000]
 [    33  58000]
 [    42 142000]
 [    29  90000]
 [    29  92000]
 [    25  88000]
 [    40  99000]
 [    38  79000]
 [    33  48000]
 [    28 102000]
 [    44  94000]
 [    33  99000]
 [    28  86000]
 [    33  75000]
 [    31  57000]
 [    43  54000]
 [    44  66000]
 [    20  20000]
 [    28  57000]
 [    45  38000]
 [    46  15000]
```

[31 20000]
[38 124000]
[39 77000]
[53 29000]
[33 23000]
[27 80000]
[60 84000]
[29 55000]
[49 75000]
[32 68000]
[48 53000]
[60 42000]
[47 83000]
[42 54000]
[39 123000]
[51 77000]
[42 122000]
[55 128000]
[53 32000]
[37 64000]
[38 91000]
[25 40000]
[24 71000]
[34 104000]
[36 46000]
[26 73000]
[57 40000]
[41 110000]
[18 64000]
[32 40000]
[22 142000]
[60 30000]
[56 36000]
[34 65000]
[35 77000]
[39 81000]
[30 80000]
[54 83000]
[40 32000]
[19 101000]
[33 78000]
[35 75000]
[35 76000]
[41 128000]
[34 67000]
[44 128000]
[56 84000]
[30 103000]
[35 109000]
[21 67000]
[53 91000]
[25 47000]
[50 90000]
[42 124000]

[44 16000]
[32 97000]
[40 79000]
[42 103000]
[53 44000]
[38 115000]
[43 149000]
[33 73000]
[24 15000]
[40 21000]
[33 85000]
[39 95000]
[30 63000]
[43 60000]
[18 25000]
[45 60000]
[51 67000]
[38 41000]
[28 57000]
[42 34000]
[18 101000]
[31 84000]
[41 114000]
[34 107000]
[41 136000]
[49 41000]
[41 41000]
[33 60000]
[27 81000]
[44 58000]
[38 150000]
[44 68000]
[42 41000]
[49 82000]
[34 101000]
[22 116000]
[22 15000]
[32 85000]
[39 67000]
[30 100000]
[28 115000]
[53 74000]
[34 38000]
[44 51000]
[29 15000]
[35 64000]
[45 92000]
[59 67000]
[27 27000]
[37 81000]
[47 95000]
[39 20000]
[28 76000]
[35 82000]

[34 33000]
[29 78000]
[40 131000]
[24 55000]
[40 15000]
[31 48000]
[42 15000]
[23 85000]
[45 78000]
[38 79000]
[32 56000]
[54 150000]
[34 60000]
[43 19000]
[39 58000]
[24 15000]
[57 64000]
[41 40000]
[34 15000]
[37 15000]
[32 15000]
[24 98000]
[42 84000]
[44 112000]
[32 55000]
[33 55000]
[47 15000]
[27 38000]
[35 34000]
[28 88000]
[37 93000]
[25 16000]
[27 66000]
[28 119000]
[26 31000]
[36 79000]
[46 15000]
[27 15000]
[56 87000]
[34 35000]
[49 38000]
[51 91000]
[41 15000]
[55 128000]
[52 70000]
[38 25000]
[56 100000]
[35 33000]
[24 87000]
[48 106000]
[29 33000]
[49 65000]
[40 63000]
[31 80000]

[40 31000]
[41 30000]
[38 71000]
[42 123000]
[30 39000]
[34 134000]
[37 74000]
[30 102000]
[47 71000]
[42 15000]
[22 68000]
[23 56000]
[40 35000]
[18 69000]
[37 77000]
[26 79000]
[18 15000]
[38 59000]
[28 86000]
[36 124000]
[35 26000]
[42 88000]
[50 118000]
[34 40000]
[36 133000]
[38 70000]
[55 80000]
[47 99000]
[43 110000]
[18 83000]
[42 86000]
[32 146000]
[54 74000]
[23 61000]
[38 60000]
[27 34000]
[23 85000]
[45 82000]
[41 59000]
[56 114000]
[46 74000]
[30 55000]
[41 92000]
[25 119000]
[27 35000]
[44 81000]
[44 15000]
[37 105000]
[32 86000]
[32 97000]
[48 93000]
[32 88000]
[24 63000]
[53 75000]

[42 64000]
[31 67000]
[53 67000]
[21 38000]
[60 50000]
[42 72000]
[23 17000]
[38 97000]
[50 123000]
[49 90000]
[26 93000]
[46 120000]
[35 73000]
[41 36000]
[30 61000]
[36 61000]
[35 46000]
[38 71000]
[56 15000]
[45 86000]
[48 76000]
[45 73000]
[18 21000]
[22 36000]
[56 56000]
[36 36000]
[29 75000]
[46 42000]
[23 59000]
[54 104000]
[58 136000]
[26 25000]
[36 47000]
[60 80000]
[36 45000]
[18 29000]
[51 83000]
[45 50000]
[32 31000]
[38 81000]
[43 91000]
[55 51000]
[33 15000]
[53 42000]
[31 96000]
[48 73000]
[37 66000]
[44 122000]
[41 48000]
[41 62000]
[18 97000]
[37 138000]
[33 56000]
[21 47000]

[38 15000]
[18 36000]
[34 29000]
[43 132000]
[22 110000]
[21 77000]
[21 65000]
[47 79000]
[42 92000]
[42 84000]
[60 81000]
[35 89000]
[45 135000]
[33 61000]
[19 99000]
[51 112000]
[46 83000]
[43 76000]
[50 15000]
[38 112000]
[28 47000]
[18 133000]
[44 22000]
[32 50000]
[44 69000]
[21 112000]
[60 103000]
[35 83000]
[28 98000]
[35 38000]
[40 67000]
[49 47000]
[53 65000]
[34 49000]
[36 56000]
[38 130000]
[30 40000]
[24 23000]
[39 108000]
[36 141000]
[55 85000]
[46 42000]
[41 76000]
[26 57000]
[25 94000]
[51 81000]
[18 68000]
[43 100000]
[40 45000]
[24 15000]
[33 106000]
[44 88000]
[25 138000]
[28 121000]

```
[    40  66000]
[    29  88000]
[    30  50000]
[    30  96000]
[    33  74000]
[    31  30000]
[    21  70000]
[    22  27000]
[    18  81000]
[    30  65000]
[    47  36000]
[    36  45000]
[    41  78000]
[    49 112000]
[    52  87000]
[    41  61000]
[    30  88000]
[    30  89000]
[    50  76000]
[    41 106000]
[    39  69000]
[    48  70000]
[    43  15000]
[    36 109000]
[    32 113000]
[    21  83000]
[    39 150000]
[    50  46000]
[    40  46000]
[    42  62000]
[    30  60000]
[    32 111000]
[    40  65000]
[    21  23000]
[    42  54000]
[    40  31000]
[    40 134000]
[    48  64000]
[    36  35000]
[    32  90000]
[    31 135000]
[    46  41000]
[    51  23000]
[    50  20000]
[    36  33000]
[    49  36000]]
[0 0 0 0 0 1 1 0 1 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 1 0 0 1 0 0 1 0 1 0 0 1
 0 1 0 1 1 1 0 0 1 1 1 1 1 1 0 0 0 1 0 1 0 0 0 0 1 1 0 1 0 0 1 0 0 0 0 0 1
 0 1 1 1 1 0 1 0 1 1 1 0 1 1 0 1 1 0 0 0 1 0 0 1 0 0 1 0 0 1 0 0 1 1 1 0 0
 0 0 0 1 1 0 1 0 0 0 0 1 0 1 1 0 0 0 1 0 1 0 0 1 1 0 0 0 0 1 0 0 0 1 0 1 0
 0 1 0 0 1 0 1 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 1 0 0 1 0 1 1 1 1 1 1 1
 0 0 1 0 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 1 0 1 0 1 0
 1 1 1 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 1 0 0 1 1 0 1 0 0 0 0 0 1 1 0 1 0
 1 1 0 0 0 1 1 0 0 0 0 1 0 0 1 0 1 1 0 0 1 0 0 0 0 0 0 1 1 0 1 0 1 0 0 1 0
```

```
 1 1 0 0 0 0 0 1 0 0 0 0 0 1 1 0 1 0 0 1 0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 1 1
 0 0 1 0 0 1 0 1 1 1 0 0 1 0 1 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1
 0 0 1 1 0 1 0 1 0 0 1 1 0 1 0 1 0 0 1 0 1 0 0 0 1 1 1 1 0 1]
0.84375
0.7375
              precision    recall  f1-score   support

           0       0.83      0.90      0.86       250
           1       0.80      0.70      0.75       150

    accuracy                           0.82       400
   macro avg       0.82      0.80      0.81       400
weighted avg       0.82      0.82      0.82       400
```

EXPERIMENT NO: 20

Experiment to understand K-means clustering algorithm for a given data set.

Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df=pd.read_csv('Mall_Customers.csv')
print(df.info())
print(df.head())
sns.pairplot(df)
features=df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
print(Final.head())
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8).map(plt.scatter,"Annual Income (k$)","Spending Score (1-100)").add_legend();
plt.show()
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
```

```
  model=KMeans(n_clusters=i)
  model.fit(features_el)
  wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

Output:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 200 entries, 0 to 199

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	CustomerID	200 non-null	int64
1	Gender	200 non-null	object
2	Age	200 non-null	int64
3	Annual Income (k		

$)200non - nullint644SpendingScore(1 - 100)200non - nullint64dtypes: int64(4), object(1)memo$

) Spending Score (1-100)

0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

/tmp/ipykernel_4411/1786908403.py:15: SettingWithCopyWarning:

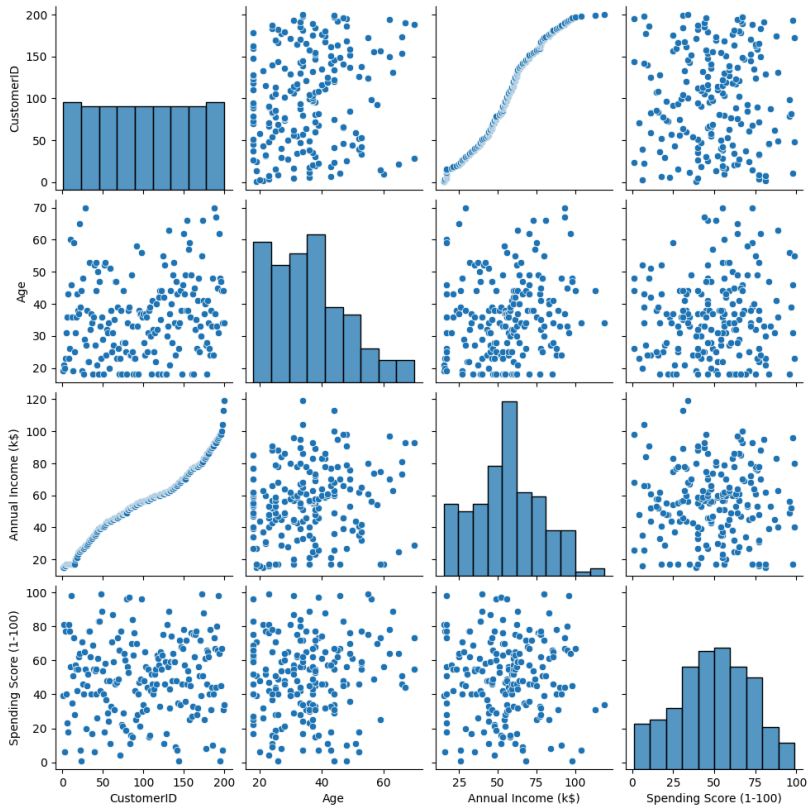
A value is trying to be set on a copy of a slice from a DataFrame.

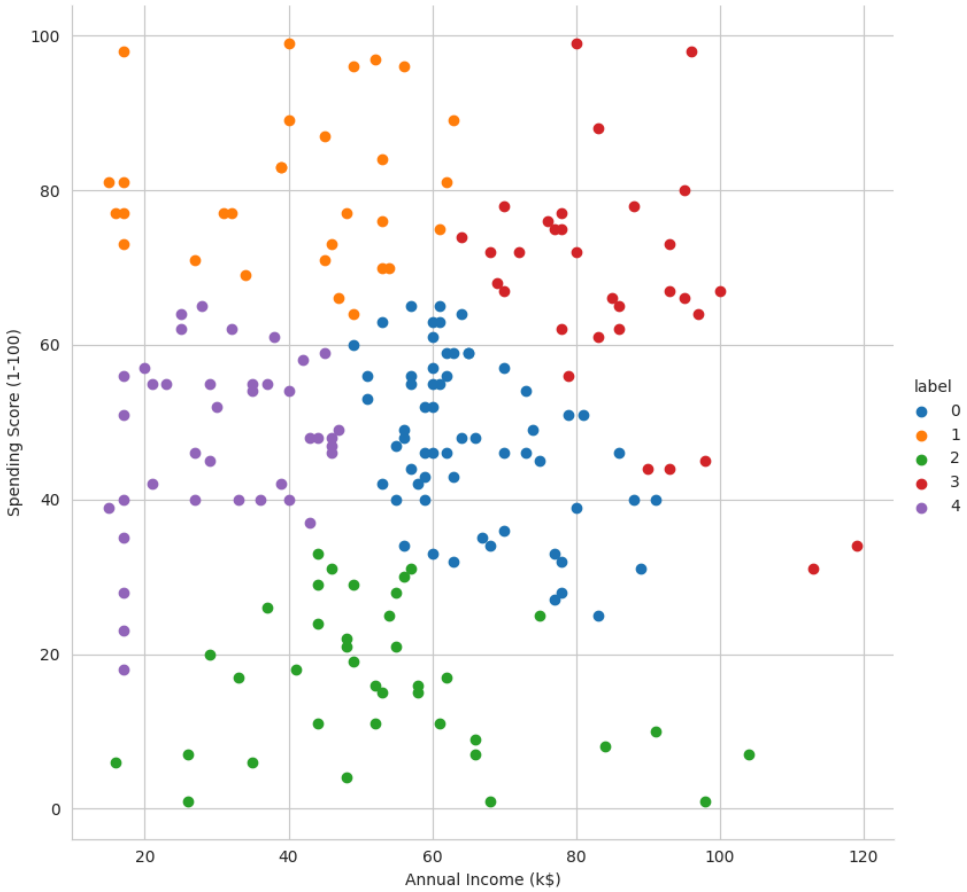
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
Final['label']=model.predict(features)
```

	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	4
1	15	81	1
2	16	6	2
3	16	77	1
4	17	40	4





[<matplotlib.lines.Line2D at 0x7e90cd081810>]

